

MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Tarea 3 - Módulo 8



Campus Internacional
CIBERSEGURIDAD

ENIIT
INNOVATION BUSINESS SCHOOL



UCAM
UNIVERSIDAD
CATÓLICA DE MURCIA

• 1. Entendiendo que pide el ejercicio.....	3
◦ 1.1 Descargamos APK de Diva.....	3
◦ 1.2 Extraemos el código fuente de la app.....	3
• 2. Análisis estático los retos.....	6
◦ 2.1 Insecure Logging.....	6
◦ 2.2 Hardcoding Issues.....	11
◦ 2.3 Insecure Data Storage — Shared Preferences.....	14
◦ 2.4 Insecure Data Storage — Part 2.....	17
◦ 2.5 Insecure Data Storage — Part 3.....	20
◦ 2.6 Insecure Data Storage — Part 4.....	24
◦ 2.7 Input Validation Issues — Part 1.....	27
◦ 2.8 Input Validation Issues — Part 2.....	31
◦ 2.9: Access Control Issues — Part 1.....	34
◦ 2.10 Access Control Issues — Part 2.....	39
◦ 2.11 Access Control Issues — Part 3.....	43
◦ 2.12 Hardcoding Issues — Part 2.....	48
◦ 2.13 Input Validation Issues — Part 3.....	53

1. Entendiendo que pide el ejercicio

En el siguiente enlace puedes acceder al ejercicio que tienes que realizar.

Contiene tanto los binarios para su análisis como una guía de su realización:

[Análisis de vulnerabilidades en aplicaciones Android \(1\) - Jaymon security](#)

En este ejercicio se realiza un **análisis estático del código de la aplicación DIVA**, con el objetivo de identificar dónde se encuentra cada vulnerabilidad dentro del código decompilado y documentar su causa técnica.

Los retos de DIVA **ya fueron resueltos y explicados en el [Ejercicio 2: How to crack the challenges of DIVA](#)**, por lo que en este apartado no se pretende repetir todos los pasos necesarios para superarlos. En su lugar, el análisis se centra en **revisar las clases, métodos y ficheros relevantes de la aplicación para localizar los patrones de programación inseguros que originan cada vulnerabilidad**.

Por tanto, el objetivo principal no es únicamente demostrar que los retos pueden resolverse, sino explicar por qué la aplicación es vulnerable, qué parte del código provoca el fallo y qué medidas podrían aplicarse para mitigarlo.

1.1 Descargamos APK de Diva

Descargamos la app [DIVA](#) y seguiremos los pasos de [jaymonsecurity](#) para extraer e instalar en el emulador.

Como ya tenemos la aplicación instalada en un dispositivo android 7 de las prácticas anteriores, nos saltamos el paso de la instalación.

1.2 Extraemos el código fuente de la app

La guía usa la herramienta [dex2jar](#) para convertir el APK de DIVA en un [.jar](#).

Instalamos esta herramienta:

```
sudo apt install dex2jar
```

Comprobamos que funciona:

```
d2j-dex2jar -h
```

```
(xxniwexx@kali)-[~/Escritorio]
$ d2j-dex2jar -h
d2j-dex2jar -- convert dex to jar
usage: d2j-dex2jar [options] <file0> [file1 ... fileN]
options:
  --skip-exceptions      skip-exceptions
  -d,--debug-info        translate debug info
  -e,--exception-file <file> detail exception file, default is $current_dir/[file-name]-error.zip
  -f,--force             force overwrite
  -h,--help              Print this help message
  -n,--not-handle-exception not handle any exceptions thrown by dex2jar
  -nc,--no-code
  -o,--output <out-jar-file> output .jar file, default is $current_dir/[file-name]-dex2jar.jar
  -os,--optimize-synchronized optimize-synchronized
  -p,--print-ir          print ir to System.out
  -r,--reuse-reg         reuse register while generate java .class file
  -s                     same with --topological-sort/-ts
  -ts,--topological-sort sort block by topological, that will generate more readable code, default enabled
version: reader-2.1-SNAPSHOT, translator-2.1-SNAPSHOT, ir-2.1-SNAPSHOT
```

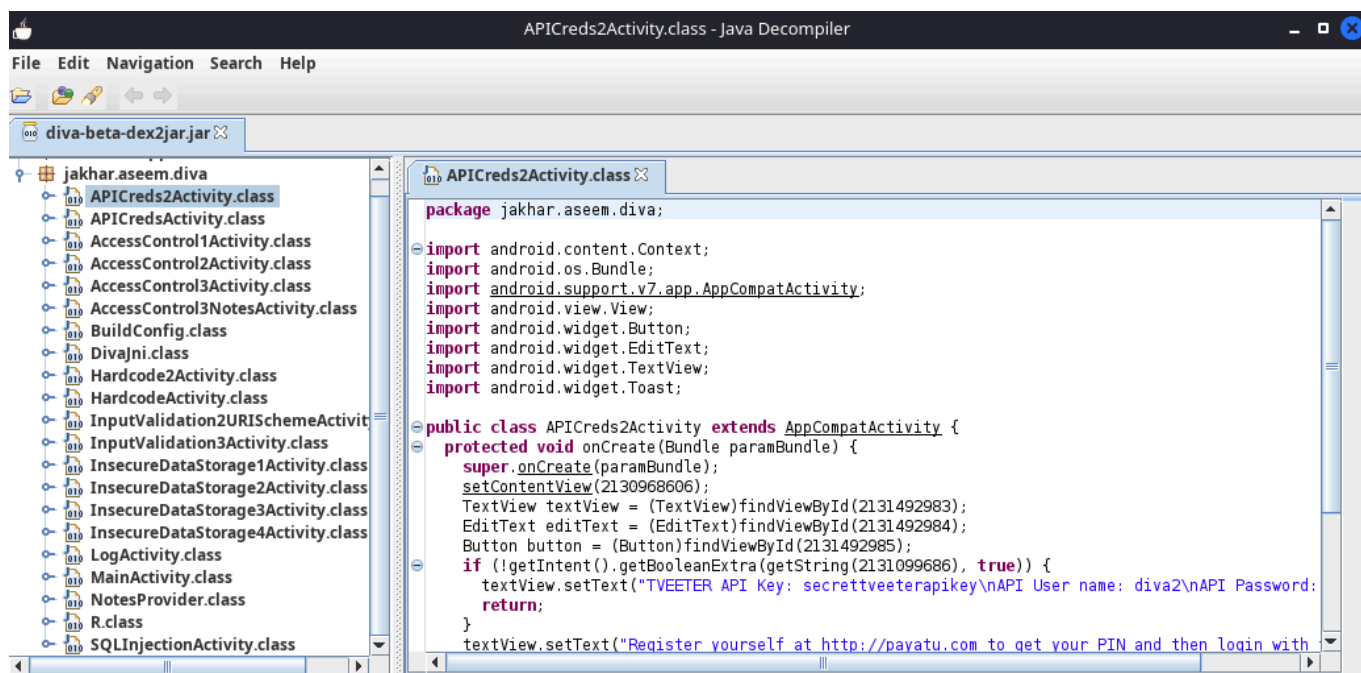
Convertimos el APK:

```
d2j-dex2jar -f diva-beta.apk
```

```
(xxniwexx@kali)-[~/Escritorio]
$ d2j-dex2jar -f diva-beta.apk
dex2jar diva-beta.apk -> ./diva-beta-dex2jar.jar
```

Abrimos el resultado con JD-GUI:

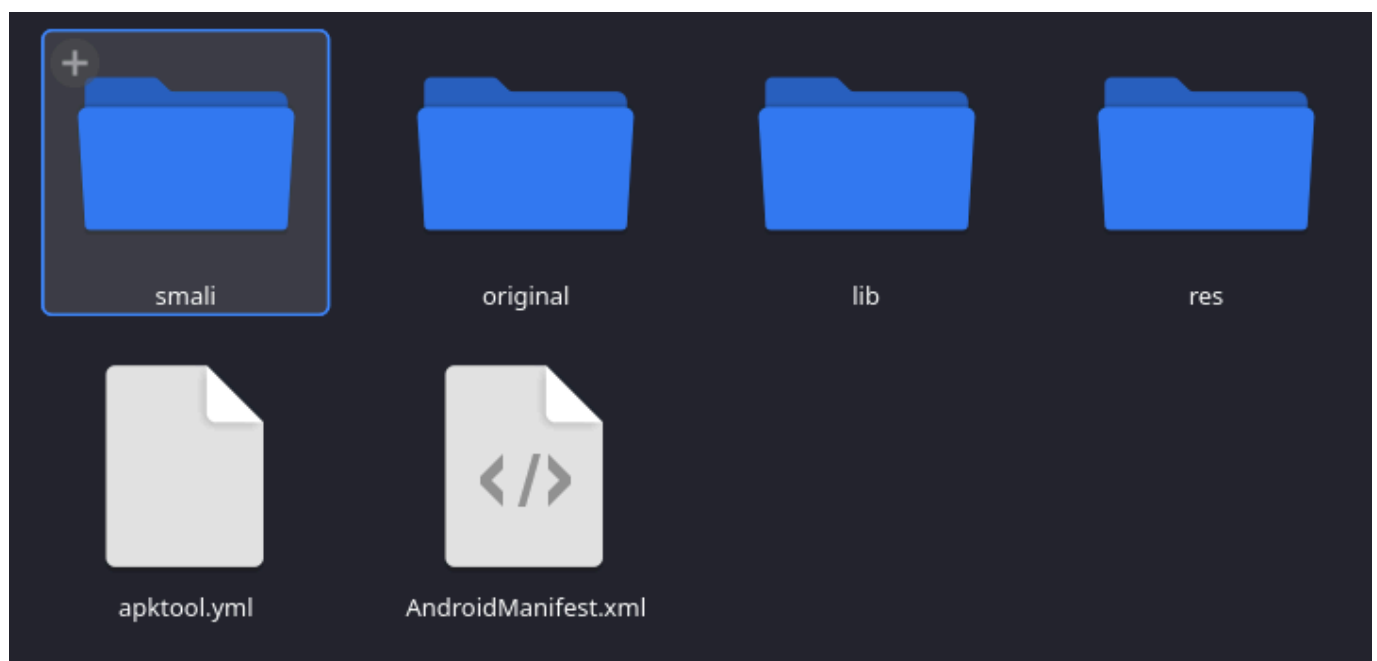
```
jd-gui diva-beta-dex2jar.jar
```



Extraemos el **AndroidManifest.xml** de DIVA con **apktool**:

```
apktool d diva-beta.apk
```

```
$ apktool d diva-beta.apk
I: Using Apktool 2.7.0-dirty on diva-beta.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/xxniwexx/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
```



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```

▼<manifest xmlns:android="http://schemas.android.com/apk/res/android" package="jakhar.aseem.diva"
platformBuildVersionCode="23" platformBuildVersionName="6.0-2166767">
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  ▼<application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher"
android:label="@string/app_name" android:supportRtl="true" android:theme="@style/AppTheme">
    ▼<activity android:label="@string/app_name" android:name="jakhar.aseem.diva.MainActivity"
android:theme="@style/AppTheme.NoActionBar">
      ▼<intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <activity android:label="@string/d1" android:name="jakhar.aseem.diva.LogActivity"/>
    <activity android:label="@string/d2" android:name="jakhar.aseem.diva.HardcodeActivity"/>
    <activity android:label="@string/d3" android:name="jakhar.aseem.diva.InsecureDataStorage1Activity"/>
    <activity android:label="@string/d4" android:name="jakhar.aseem.diva.InsecureDataStorage2Activity"/>
    <activity android:label="@string/d5" android:name="jakhar.aseem.diva.InsecureDataStorage3Activity"/>
    <activity android:label="@string/d6" android:name="jakhar.aseem.diva.InsecureDataStorage4Activity"/>
    <activity android:label="@string/d7" android:name="jakhar.aseem.diva.SQLInjectionActivity"/>
    <activity android:label="@string/d8" android:name="jakhar.aseem.diva.InputValidation2URISchemeActivity"/>
    <activity android:label="@string/d9" android:name="jakhar.aseem.diva.AccessControl1Activity"/>
    ▼<activity android:label="@string/apic_label" android:name="jakhar.aseem.diva.APICredsActivity">
      ▼<intent-filter>
        <action android:name="jakhar.aseem.diva.action.VIEW_CREDS"/>
        <category android:name="android.intent.category.DEFAULT"/>
      </intent-filter>
    </activity>
    <activity android:label="@string/d10" android:name="jakhar.aseem.diva.AccessControl2Activity"/>
    ▼<activity android:label="@string/apic2_label" android:name="jakhar.aseem.diva.APICreds2Activity">
      ▼<intent-filter>
        <action android:name="jakhar.aseem.diva.action.VIEW_CREDS2"/>
        <category android:name="android.intent.category.DEFAULT"/>
      </intent-filter>
    </activity>
    <provider android:authorities="jakhar.aseem.diva.provider.notesprovider" android:enabled="true" android:exported="true"
android:name="jakhar.aseem.diva.NotesProvider"/>
  </application>
</manifest>

```

AndroidManifest.xml es clave para analizar vulnerabilidades como por ejemplo, Access Control o identificar Activities o Providers accesibles desde fuera de la app.

2. Análisis estático los retos

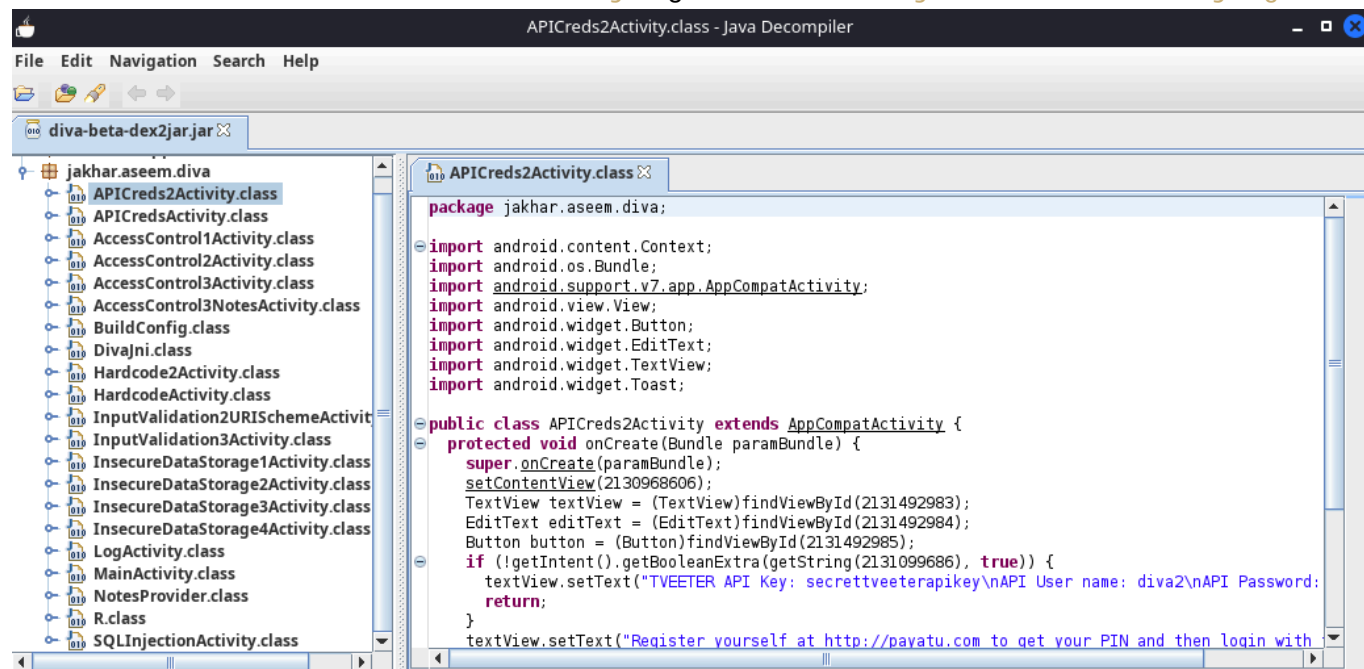
2.1 Insecure Logging

Vamos a demostrar dónde está el fallo en el código Java y por qué es vulnerable.

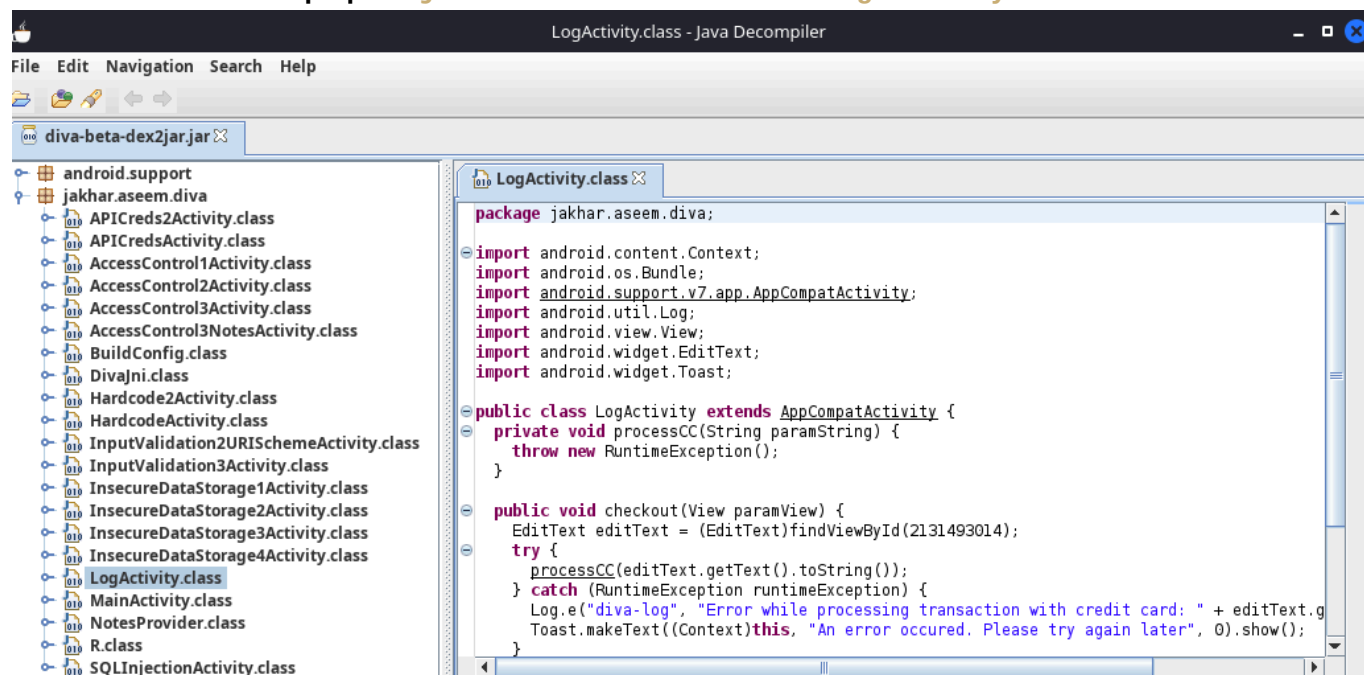
DIVA propone este reto como el primer challenge, **Insecure Logging**, y el código vulnerable está en **LogActivity.java**. En esa clase, el método **checkout()** lee el número introducido en el campo de tarjeta y, cuando se produce una excepción, lo escribe en el log con la etiqueta **diva-log**.

Objetivo del reto: El objetivo es identificar si la aplicación registra información sensible en los logs del sistema Android. En este caso, el dato sensible es el número de tarjeta de crédito introducido por el usuario. Como ya se analizó y resolvió en el [Reto 1 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: La aplicación registra información sensible en los logs. Identificaremos que esos datos acaban siendo visibles en **Logcat**.

Localizamos la clase vulnerable: Abrimos el **.jar** generado con **dex2jar** con la herramienta **jd-gui**:



Buscamos dentro del paquete **jakhar.aseem.diva** la clase **LogActivity**:



Buscamos el método **checkout(View paramView)**:

```

public class LoginActivity extends AppCompatActivity {
    private void processCC(String paramString) {
        throw new RuntimeException();
    }

    public void checkout(View paramView) {
        EditText editText = (EditText)findViewById(2131493014);
        try {
            processCC(editText.getText().toString());
        } catch (RuntimeException runtimeException) {
            Log.e("diva-log", "Error while processing transaction with credit card: " + editText.getText().toString());
            Toast.makeText((Context)this, "An error ocured. Please try again later", 0).show();
        }
    }
}

```

donde:

- En este método se encuentra la llamada vulnerable a **Log.e()** que registra el número de tarjeta introducido por el usuario.

El código vulnerable: El fallo está en el método:

```
checkout(View paramView)
```

La parte importante es:

```

EditText cctxt = (EditText) findViewById(R.id.ccText);

try {
    processCC(cctxt.getText().toString());
} catch (RuntimeException re) {
    Log.e("diva-log", "Error while processing transaction with credit card:
"
        + cctxt.getText().toString());

    Toast.makeText(this, "An error ocured. Please try again later",
        Toast.LENGTH_SHORT).show();
}

```

El repositorio oficial muestra que **checkout()** obtiene el texto del campo **ccText**, llama a **processCC()**, captura una **RuntimeException** y después registra el número de tarjeta con **Log.e("diva-log", ...)**.

El fallo explicado: El código:

```
Log.e("diva-log", "Error while processing transaction with credit card: "
    + cctxt.getText().toString());
```

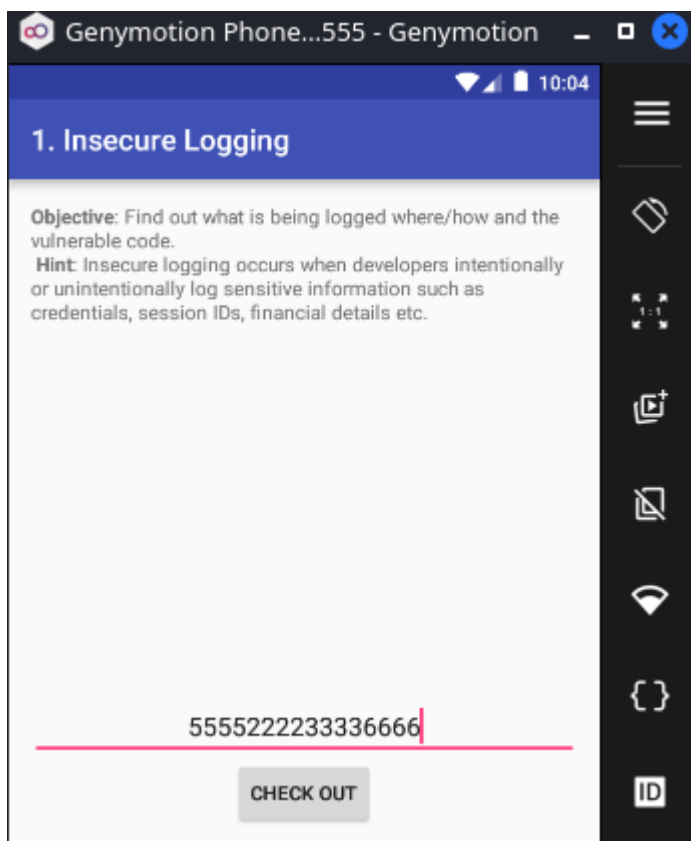
donde:

- La aplicación está concatenando directamente el valor introducido por el usuario:
`cctxt.getText().toString()` con un mensaje de error que se envía a `Logcat`.

El patrón vulnerable es: `Dato sensible del usuario` → `concatenación en string` → `Log.e()`.

Comprobamos en el dispositivo android:

```
adb logcat
```



Nueva pestaña

Dividir vista

Copiar

Pegar

```
05-04 10:03:59.264 578 780 I WindowManager: WIN DEATH: Window{6ec05df u0 com.withsecure.dz/com.WithSecure.dz.activities.MainActivity
05-04 10:03:59.264 578 780 W InputDispatcher: Attempted to unregister already unregistered input channel '6ec05df com.withsecure.dz/
.dz.activities.MainActivity (server)'
05-04 10:03:59.268 578 781 D ActivityManager: cleanUpApplicationRecord -- 1128
05-04 10:03:59.274 578 1116 D GraphicsStats: Buffer count: 3
05-04 10:03:59.274 578 1161 D ActivityManager: cleanUpApplicationRecord -- 2659
05-04 10:03:59.274 578 1161 W ActivityManager: Scheduling restart of crashed service com.withsecure.dz/com.WithSecure.dz.services.Cli
9041738ms
05-04 10:03:59.274 578 1161 W ActivityManager: Scheduling restart of crashed service com.withsecure.dz/com.WithSecure.dz.services.Ser
9051738ms
05-04 10:04:00.565 578 948 E ActivityManager: applyOptionsLocked: Unknown animationType=0
05-04 10:04:00.929 578 599 E eglCodecCommon: goldfish_dma_create_region: could not obtain fd to device! fd -1 errno=2
05-04 10:04:01.855 578 948 I ActivityManager: START u0 {cmp=jakhar.aseem.diva/.LogActivity} from uid 10065 on display 0
05-04 10:04:01.955 578 599 I ActivityManager: Displayed jakhar.aseem.diva/.LogActivity: +93ms
05-04 10:04:01.973 1643 1660 D OpenGLRenderer: endAllActiveAnimators on 0xd2998400 (RippleDrawable) with handle 0xd1ed5c50
05-04 10:04:05.035 303 327 W genymotion_audio: Not supplying enough data to HAL, expected position 1812394 , only wrote 1517040
05-04 10:05:18.847 1643 1643 E diva-log: Error while processing transaction with credit card: 555522233336666
05-04 10:05:18.852 303 326 W genymotion_audio: Not supplying enough data to HAL, expected position 1517211 , only wrote 1517040
05-04 10:05:20.855 578 578 W WindowManager: Attempted to remove non-existing token: android.os.Binder@f3cd84d
05-04 10:05:22.077 303 327 W genymotion_audio: Not supplying enough data to HAL, expected position 1824539 , only wrote 1671840
```

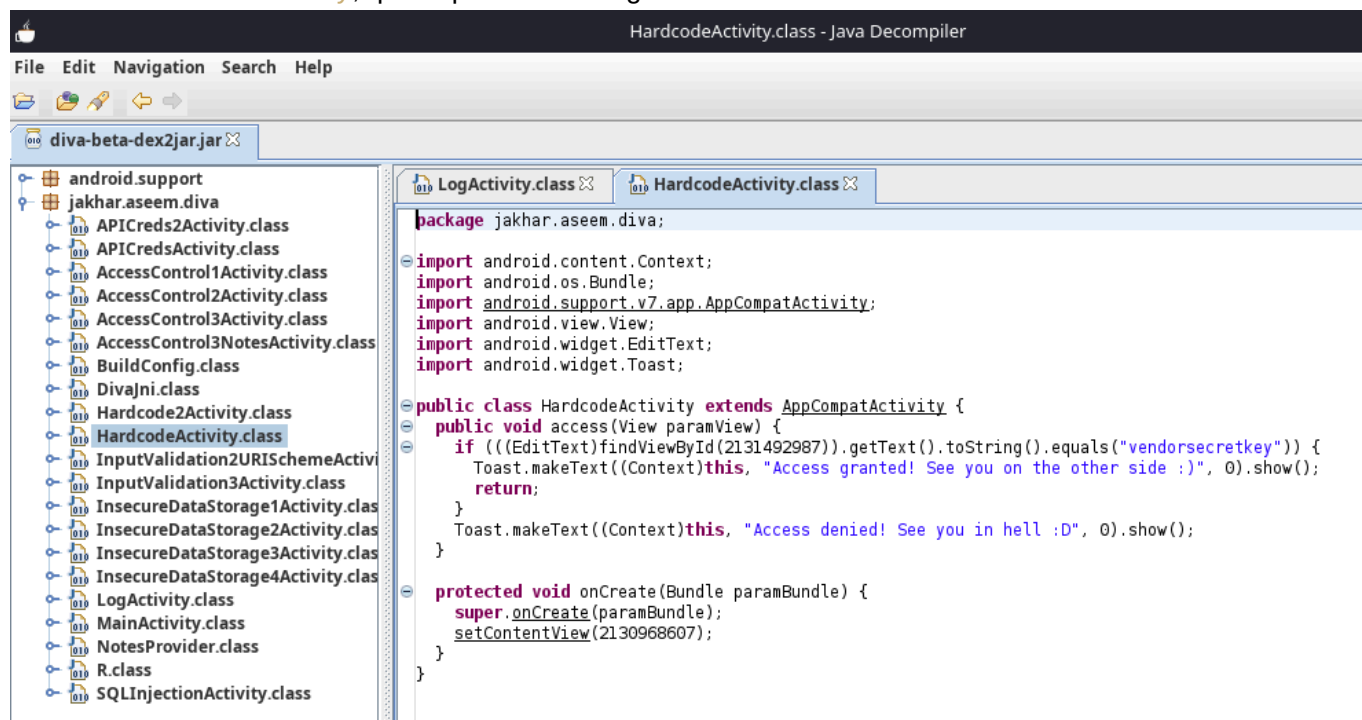
Tabla resumen del reto 1:

Campo	Resultado
Reto	Insecure Logging
Clase vulnerable	LogActivity.java
Método vulnerable	checkout(View view)
Campo leído	R.id.ccText
Dato sensible expuesto	Número de tarjeta de crédito
API problemática	Log.e()
Etiqueta de log	diva-log
Tipo de fallo	Exposición de información sensible en logs
Causa raíz	Se concatena la entrada del usuario en un mensaje de log
Fragmento vulnerable	Log.e("diva-log", "... credit card: " + cctxt.getText().toString())
Cómo se comprueba	Ejecutando adb logcat -s diva-log e introduciendo una tarjeta en la app
Impacto	Un atacante con acceso a logs podría leer información sensible
Mitigación	No registrar datos sensibles. Eliminar logs en producción o enmascarar la información

2.2 Hardcoding Issues

Como ya se analizó y resolvió en el [Reto 2 de la Practica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Confiar en un secreto almacenado en el cliente. Identificaremos una clave secreta hardcodeada dentro del código de la aplicación.

Buscamos la clase `HardcodeActivity`: Dentro del paquete `jakhar.aseem.diva`, vamos a localizar la clase `HardcodeActivity`, que implementa la lógica de este reto.



Buscamos el método vulnerable `access(View view)`: En el código de DIVA, esta clase obtiene el texto introducido por el usuario desde el campo `hcKey` y lo compara directamente con una cadena fija. El código vulnerable:

```
public void access(View view) {
    EditText hkey = (EditText) findViewById(R.id.hcKey);

    if (hkey.getText().toString().equals("vendorsecretkey")) {
        Toast.makeText(this, "Access granted! See you on the other side
:)",
            Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Access denied! See you in hell :D",
            Toast.LENGTH_SHORT).show();
    }
}
```

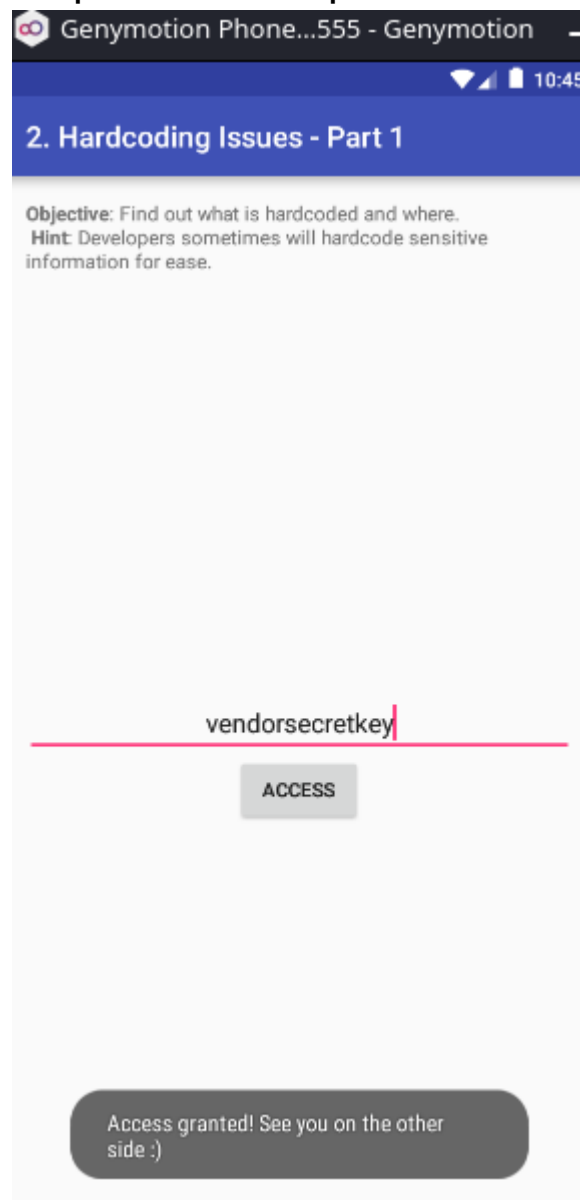
donde:

- Se obtiene el texto introducido en el campo `hcKey`.
- Se convierte ese texto a String.

- Se compara con la cadena fija `vendorsecretkey`.
- Si coincide, muestra el mensaje de acceso concedido.

El fallo de seguridad es que la clave de acceso está almacenada dentro del propio APK. Aunque el usuario normal no la vea en la interfaz, cualquier persona que decompile la aplicación con herramientas como `dex2jar`, `JD-GUI` o `JADX` puede leerla.

Comprobamos en el dispositivo android:



La causa de esta vulnerabilidad es confiar en un secreto almacenado en el cliente. En una aplicación real, este patrón permitiría extraer claves API, tokens, contraseñas internas o secretos de cifrado. La mitigación consiste en no guardar secretos dentro del APK y trasladar la validación a un servidor o a un mecanismo seguro que no exponga la clave en el código cliente.

Tabla resumen del reto 2:

Campo	Resultado
Reto	Hardcoding Issues - Part 1
Clase vulnerable	<code>HardcodeActivity.java</code>
Método vulnerable	<code>access(View view)</code>
Campo leído	<code>R.id.hcKey</code>
Secreto expuesto	<code>vendorsecretkey</code>
Tipo de fallo	Secreto hardcodeado
Causa raíz	Clave escrita directamente en código Java
Impacto	Extracción del secreto mediante decompilación
Mitigación	No almacenar secretos en cliente. Validar en servidor

2.3 Insecure Data Storage — Shared Preferences

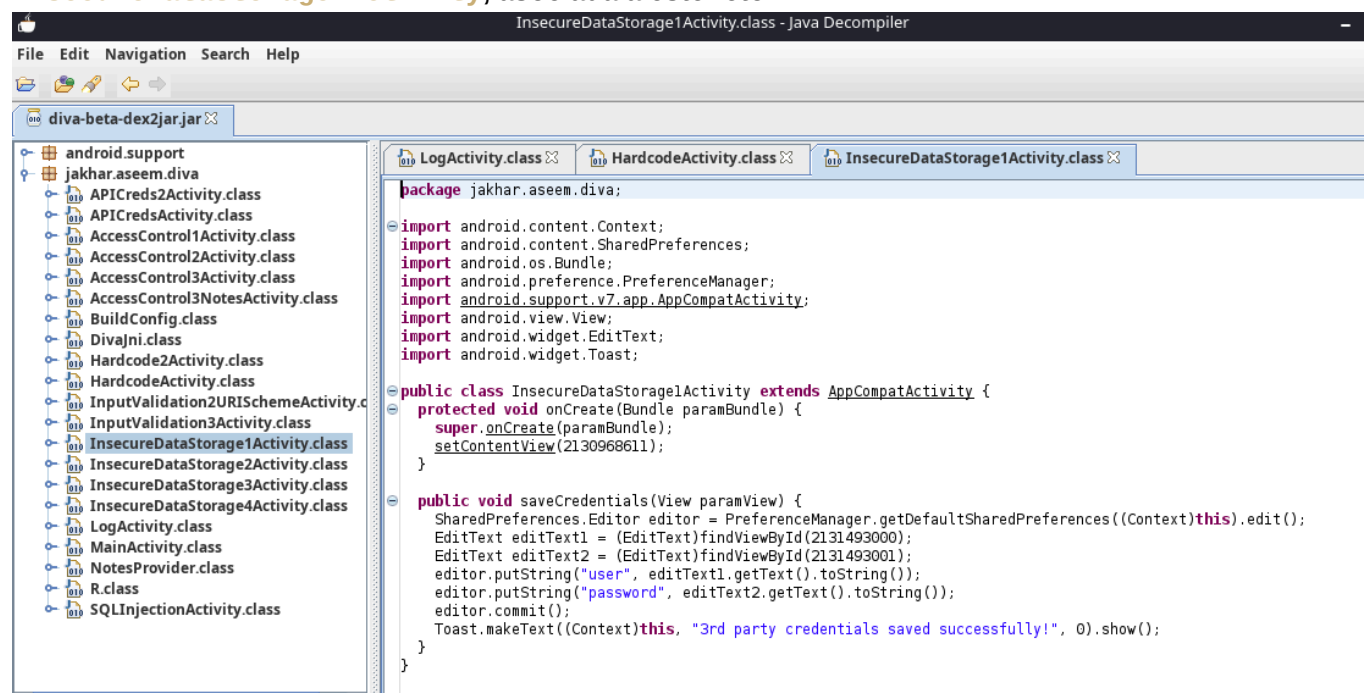
Como ya se analizó y resolvió en el [Reto 3 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Utilizar **SharedPreferences** como almacén de información sensible sin cifrado. Identificaremos dónde guarda la aplicación las credenciales introducidas por el usuario y comprobar si se almacenan de forma segura. En este caso, la app guarda:

Usuario
Contraseña

en un fichero XML de **SharedPreferences**.

Android explica que **SharedPreferences** se usa para guardar pares **clave-valor** en un fichero gestionado por el framework. Es decir, no es una base de datos ni un almacén seguro de credenciales por sí mismo.

Dentro del paquete **jakhar.aseem.diva**, vamos a localizar la clase **InsecureDataStorage1Activity**, asociada a este reto:



El método vulnerable **saveCredentials(View paramView)**: Este método obtiene el usuario y la contraseña desde dos campos de texto y los guarda directamente en **SharedPreferences** usando las claves **user** y **password**.

```
public void saveCredentials(View view) {
    SharedPreferences spref =
    PreferenceManager.getDefaultSharedPreferences(this);
    SharedPreferences.Editor spedit = spref.edit();

    EditText usr = (EditText) findViewById(R.id.ids1Usr);
    EditText pwd = (EditText) findViewById(R.id.ids1Pwd);

    spedit.putString("user", usr.getText().toString());
    spedit.putString("password", pwd.getText().toString());
    spedit.commit();

    Toast.makeText(this, "3rd party credentials saved successfully!",
    Toast.LENGTH_SHORT).show();
}
```

donde:

- **PreferenceManager.getDefaultSharedPreferences(this);**: Obtiene una referencia a las preferencias por defecto.
- Lee el usuario desde el campo: **R.id.ids1Usr**.
- Lee la contraseña desde el campo: **R.id.ids1Pwd**.
- **spedit.putString("user", usr.getText().toString());**: Guarda el nombre de usuario introducido en el campo de texto dentro de **SharedPreferences**, usando la clave **user**.
- **spedit.putString("password", pwd.getText().toString());**: Guarda la contraseña introducida por el usuario dentro de **SharedPreferences**, usando la clave **password**.
- Guarda los cambios con: **commit()**.

Dónde se guarda realmente: Al usar: **PreferenceManager.getDefaultSharedPreferences(this)**, android crea un fichero XML de preferencias asociado al paquete de la aplicación. En DIVA, normalmente estará en:

/data/data/jakhar.aseem.diva/shared_prefs/jakhar.aseem.diva_preferences.xml.

Comprobamos en el dispositivo android:

```
(xxniwexx@kali)-[~]
$ adb shell run-as jakhar.aseem.diva ls shared_prefs
jakhar.aseem.diva_preferences.xml

(xxniwexx@kali)-[~]
$ adb shell run-as jakhar.aseem.diva cat shared_prefs/jakhar.aseem.diva_preferences.xml

<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="user">alumno</string>
  <string name="password">password123</string>
</map>
```

La causa raíz es utilizar `SharedPreferences` como almacén de información sensible sin cifrado. Aunque este mecanismo puede ser adecuado para preferencias simples de usuario, no debe utilizarse para guardar contraseñas o secretos. La mitigación consiste en no almacenar contraseñas en el cliente y, si es necesario persistir tokens u otra información sensible, protegerlos mediante mecanismos de almacenamiento cifrado y claves gestionadas de forma segura.

Tabla resumen del reto 3:

Campo	Resultado
Reto	Insecure Data Storage - Part 1
Clase vulnerable	<code>InsecureDataStorage1Activity.java</code>
Método vulnerable	<code>saveCredentials(View view)</code>
Campos leídos	<code>R.id.ids1Usr</code> , <code>R.id.ids1Pwd</code>
API problemática	<code>SharedPreferences</code>
Método de escritura	<code>spedit.putString()</code>
Claves usadas	<code>"user"</code> , <code>"password"</code>
Fichero generado	<code>jakhar.aseem.diva_preferences.xml</code>
Ruta habitual	<code>/data/data/jakhar.aseem.diva/shared_prefs/</code>
Tipo de fallo	Almacenamiento inseguro de credenciales
Causa raíz	Usuario y contraseña guardados en texto claro
Cómo se comprueba	Leyendo el XML de <code>shared_prefs</code> con <code>adb shell</code>
Impacto	Exposición de credenciales locales
Mitigación	No guardar contraseñas. Usar almacenamiento cifrado/token seguro

2.4 Insecure Data Storage — Part 2

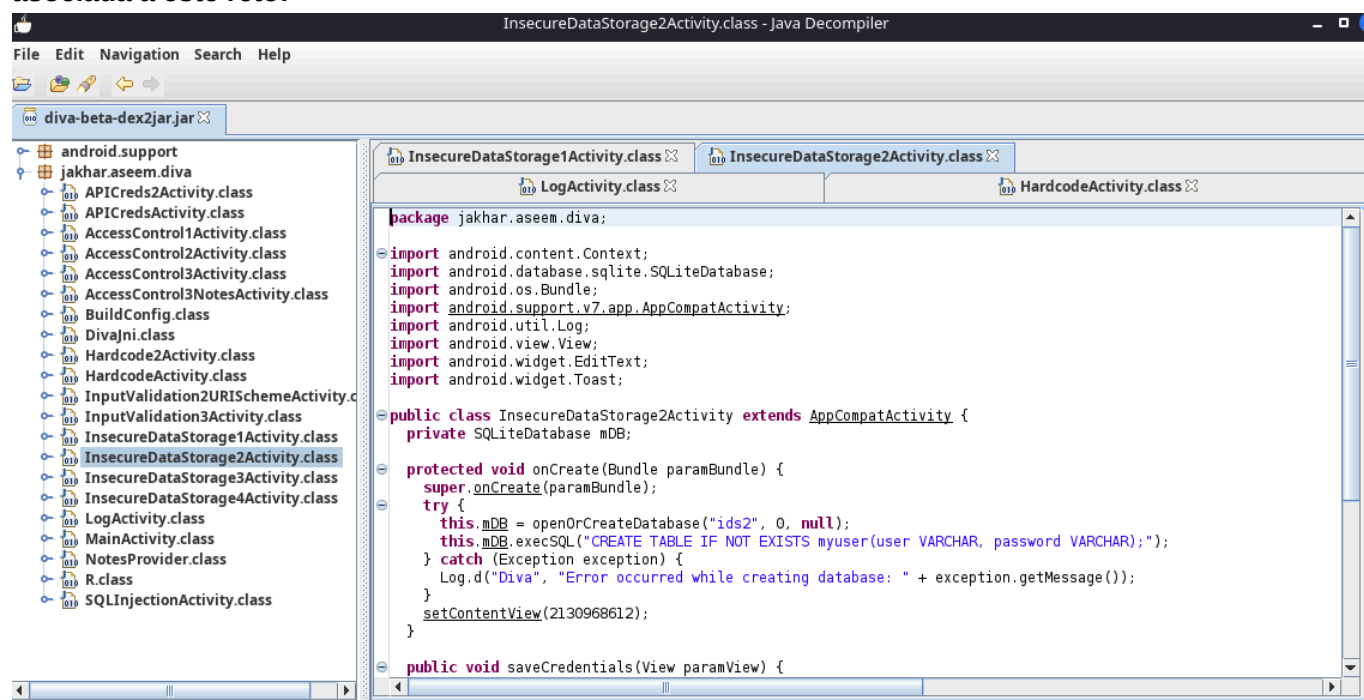
Como ya se analizó y resolvió en el [Reto 4 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Utilizar una base de datos local como almacén de credenciales sin aplicar cifrado ni protección adicional. Identificaremos dónde guarda la aplicación las credenciales introducidas por el usuario y comprobar si se almacenan de forma segura. En este caso, la app guarda:

```
Usuario
Contraseña
```

dentro de una base de datos SQLite llamada: `ids2`.

La tabla usada se llama: `myuser`.

Dentro del paquete `jakhar.aseem.diva`, localizamos la clase `InsecureDataStorage2Activity`, asociada a este reto:



Métodos relevantes: En esta clase hay dos partes importantes:

- `onCreate(Bundle savedInstanceState)`: Donde aparece la creación de la base de datos SQLite:

```
mDB = openOrCreateDatabase("ids2", MODE_PRIVATE, null);
mDB.execSQL("CREATE TABLE IF NOT EXISTS myuser(user VARCHAR, password VARCHAR);");
```

- `saveCredentials(View paramView)`: Donde se inserta las credenciales introducidas por el usuario:

```
EditText usr = (EditText) findViewById(R.id.ids2Usr);
EditText pwd = (EditText) findViewById(R.id.ids2Pwd);

mDB.execSQL("INSERT INTO myuser VALUES ('"
    + usr.getText().toString()
    + "', '"
    + pwd.getText().toString()
    + "');");
```

El código oficial de DIVA muestra que `InsecureDataStorage2Activity` crea la base de datos `ids2`, crea la tabla `myuser` (`user VARCHAR, password VARCHAR`) e inserta directamente el usuario y la contraseña mediante una sentencia `INSERT` construida por concatenación.

La vulnerabilidad: El problema principal es que las credenciales se guardan en texto claro dentro de una base de datos local. Además, hay un segundo problema de diseño: la sentencia SQL se construye concatenando directamente la entrada del usuario. En este reto el foco es el almacenamiento inseguro, pero ese patrón también es peligroso porque puede derivar en inyección SQL.

Dónde se guarda la base de datos SQLite:

- Se guarda en el directorio privado de la app: `/data/data/jakhar.aseem.diva/databases/`.
- El fichero de base de datos será normalmente:
`/data/data/jakhar.aseem.diva/databases/ids2`.
- Dentro estará la tabla: `myuser`.
- Con columnas:
 - `user`.
 - `passwor`.

Comprobamos en el dispositivo android:

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 "SELECT * FROM myuser;"'
alumno|password123
alumno4|pass4
```

La causa raíz es utilizar una base de datos local como almacén de credenciales sin aplicar cifrado ni protección adicional. Aunque el fichero se encuentre dentro del sandbox de la aplicación, un atacante con acceso al sistema de archivos, un dispositivo rooteado o un análisis forense podrían recuperar la base de datos y leer las credenciales. La mitigación consiste en no almacenar contraseñas en el cliente, usar tokens temporales cuando sea necesario y proteger cualquier dato sensible persistente mediante cifrado y claves gestionadas de forma segura.

Tabla resumen del Reto 4:

Campo	Resultado
Reto	Insecure Data Storage - Part 2
Clase vulnerable	<code>InsecureDataStorage2Activity.java</code>
Métodos relevantes	<code>onCreate()</code> , <code>saveCredentials(View view)</code>
Campos leídos	<code>R.id.ids2Usr</code> , <code>R.id.ids2Pwd</code>
API problemática	SQLite / <code>openOrCreateDatabase()</code> / <code>execSQL()</code>
Base de datos	<code>ids2</code>
Tabla	<code>myuser</code>
Columnas	<code>user</code> , <code>password</code>
Ruta habitual	<code>/data/data/jakhar.aseem.diva/databases/ids2</code>
Tipo de fallo	Almacenamiento inseguro de credenciales
Causa raíz	Usuario y contraseña guardados en SQLite en texto claro
Fragmento vulnerable	<code>INSERT INTO myuser VALUES ('" + user + "', '" + password + '");</code>
Cómo se comprueba	Consultando la tabla <code>myuser</code> con <code>sqlite3</code>
Impacto	Exposición de credenciales locales
Mitigación	No guardar contraseñas. Cifrar datos sensibles. Usar tokens seguros y consultas parametrizadas

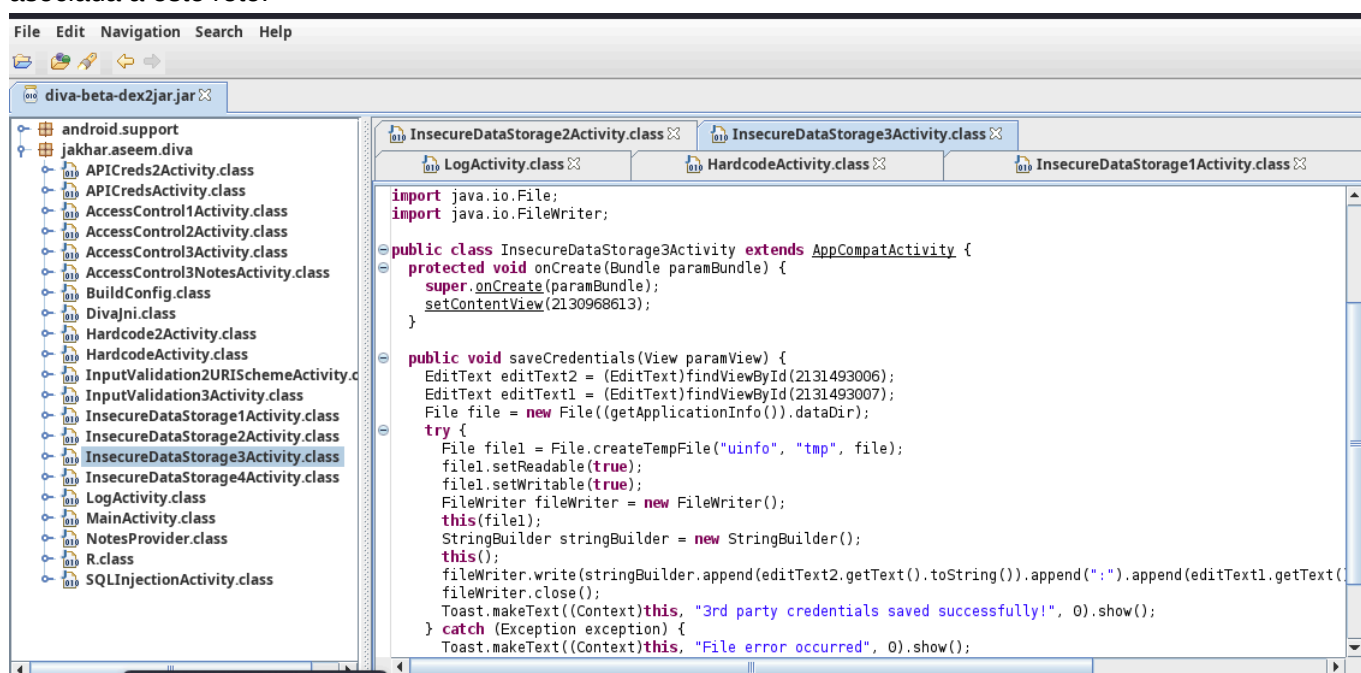
2.5 Insecure Data Storage — Part 3

Como ya se analizó y resolvió en el [Reto 5 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Usar un fichero temporal como almacén de credenciales sin cifrado ni eliminación segura posterior. En este caso, la app guarda:

Usuario
Contraseña

dentro de un fichero temporal cuyo nombre empieza por: **uinfo** y termina en: **tmp**.

Dentro del paquete `jakhar.aseem.diva`, se localiza la clase `InsecureDataStorage3Activity`, asociada a este reto:



El método vulnerable: En esta clase `insecureDataStotage3Activity`, el método `saveCredentials()` obtiene el usuario y la contraseña desde los campos `ids3Usr` e `ids3Pwd`, crea un fichero temporal en el directorio de datos de la aplicación y escribe las credenciales en texto claro.

```
public void saveCredentials(View view) {
    EditText usr = (EditText) findViewById(R.id.ids3Usr);
    EditText pwd = (EditText) findViewById(R.id.ids3Pwd);

    File ddir = new File(getApplicationInfo().dataDir);

    try {
        File uinfo = File.createTempFile("uinfo", "tmp", ddir);
        uinfo.setReadable(true);
        uinfo.setWritable(true);

        FileWriter fw = new FileWriter(uinfo);
        fw.write(usr.getText().toString() + ":" + pwd.getText().toString()
+ "\n");
        fw.close();

        Toast.makeText(this, "3rd party credentials saved successfully!",
Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        Toast.makeText(this, "File error occurred",
Toast.LENGTH_SHORT).show();
        Log.d("Diva", "File error: " + e.getMessage());
    }
}
```

donde:

- `File ddir = new File(getApplicationInfo().dataDir);`: Esta línea obtiene el directorio privado de datos de la aplicación y lo representa como un objeto `File`, para poder crear o manejar ficheros dentro de esa ubicación.
- `File uinfo = File.createTempFile("uinfo", "tmp", ddir);`: Esta línea crea un fichero temporal dentro del directorio privado de la aplicación, con un nombre que empieza por `uinfo` y termina en `tmp`.
- `fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n");`: Esta línea escribe el usuario y la contraseña en texto claro dentro del fichero temporal, separados por dos puntos.

El problema principal es que se están almacenando credenciales en un fichero local sin cifrado. Aunque el fichero esté dentro del directorio privado de la aplicación, sigue siendo una mala práctica guardar secretos en claro.

Dónde se guarda realmente: El fichero se crea dentro del directorio privado de la aplicación: `/data/data/jakhar.aseem.diva/`. Como se usa: `File.createTempFile("uinfo", "tmp", ddir);`, el nombre del fichero será aleatorio, pero tendrá una forma parecida a: `uinfo123456789tmp`. Por tanto, para localizarlo, hay que buscar ficheros que empiecen por: `uinfo`.

Comprobamos en el dispositivo android:

```

jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # cd /data/data/jakhar.aseem.diva
/data/data/jakhar.aseem.diva
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # ls
Type           Last Modified      Read Write Hidden Size  Name
-----
Directory 2026-04-23 12:41:57 GMT True True  False 4.0 KiB cache
Directory 2026-04-23 12:41:57 GMT True True  False 4.0 KiB code_cache
Directory 2026-04-23 12:41:57 GMT True False False 4.0 KiB lib
Directory 2026-05-04 15:37:42 GMT True True  False 4.0 KiB databases
Directory 2026-05-04 12:34:15 GMT True True  False 4.0 KiB files
Directory 2026-05-04 15:19:37 GMT True True  False 4.0 KiB shared_prefs
File       2026-05-05 15:41:28 GMT True True  False 14.0 B uinfo163568329tmp
File       2026-05-05 15:41:40 GMT True True  False 14.0 B uinfo1044941255tmp

Readable: True Writable: True
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # filesystem cat /data/user/0/jakhar.aseem.diva/uinfo163568329tmp
Downloading /data/user/0/jakhar.aseem.diva/uinfo163568329tmp to /tmp/tmp46wheoft.file
Streaming file from device...
Writing bytes to destination...
Successfully downloaded /data/user/0/jakhar.aseem.diva/uinfo163568329tmp to /tmp/tmp46wheoft.file
====
alumno5:pass5
====
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # filesystem cat /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp
Downloading /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp to /tmp/tmpn75gh650.file
Streaming file from device...
Writing bytes to destination...
Successfully downloaded /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp to /tmp/tmpn75gh650.file
====
alumno5:pass5
====
jakhar.aseem.diva (run) on (Android: 7.1.1) [usb] # █

```

La causa raíz es usar un fichero temporal como almacén de credenciales sin cifrado ni eliminación segura posterior. Aunque el fichero se cree dentro del directorio privado de la aplicación, en un dispositivo rooteado, en un análisis forense o en un entorno comprometido, el contenido podría recuperarse. La mitigación consiste en no almacenar contraseñas en disco, evitar persistir secretos innecesarios y utilizar mecanismos de almacenamiento cifrado cuando sea imprescindible guardar información sensible.

Tabla resumen del Reto 5:

Campo	Resultado
Reto	Insecure Data Storage - Part 3
Clase vulnerable	<code>InsecureDataStorage3Activity.java</code>
Método vulnerable	<code>saveCredentials(View view)</code>
Campos leídos	<code>R.id.ids3Usr</code> , <code>R.id.ids3Pwd</code>
API problemática	<code>File.createTempFile()</code> , <code>FileWriter</code>
Directorio usado	<code>getApplicationInfo().dataDir</code>
Ruta habitual	<code>/data/data/jakhar.aseem.diva/</code>
Nombre del fichero	<code>uinfo*tmp</code>
Formato del contenido	<code>usuario:contraseña</code>
Tipo de fallo	Almacenamiento inseguro de credenciales
Causa raíz	Usuario y contraseña escritos en texto claro en un fichero temporal
Fragmento vulnerable	<code>fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n")</code>
Cómo se comprueba	Leyendo el fichero <code>uinfo*tmp</code> desde <code>/data/data/jakhar.aseem.diva/</code>
Impacto	Exposición de credenciales locales
Mitigación	No guardar contraseñas. Evitar ficheros temporales con datos sensibles. Usar cifrado si es imprescindible

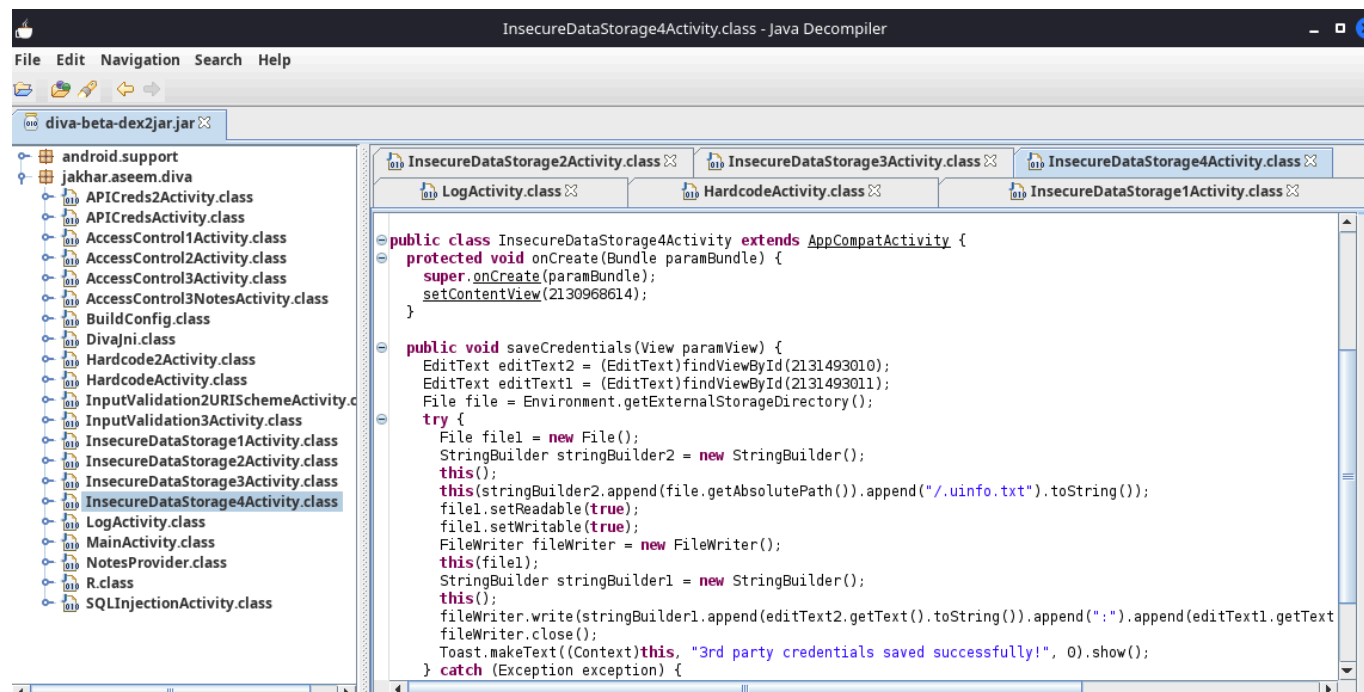
2.6 Insecure Data Storage — Part 4

Como ya se analizó y resolvió en el [Reto 6 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Usar el almacenamiento externo para guardar información sensible sin cifrado. Identificaremos que la aplicación almacena credenciales en un fichero situado fuera del directorio privado interno de la app. En este caso, la app guarda:

Usuario
Contraseña

en un fichero ubicado en el almacenamiento externo del dispositivo.

Dentro del paquete `jakhar.aseem.diva`, localizamos la clase `InsecureDataStorage4Activity`, asociada a este reto:



El método vulnerable **saveCredentials(View paramView)**: El método **saveCredentials(View paramView)** obtiene el usuario y la contraseña desde los campos **ids4Usr** e **ids4Pwd**, localiza el almacenamiento externo mediante **Environment.getExternalStorageDirectory()** y escribe las credenciales en el fichero **.uinfo.txt**.

```
public void saveCredentials(View view) {
    EditText usr = (EditText) findViewById(R.id.ids4Usr);
    EditText pwd = (EditText) findViewById(R.id.ids4Pwd);

    File sdir = Environment.getExternalStorageDirectory();

    try {
        File uinfo = new File(sdir.getAbsolutePath() + "/.uinfo.txt");
        uinfo.setReadable(true);
        uinfo.setWritable(true);

        FileWriter fw = new FileWriter(uinfo);
        fw.write(usr.getText().toString() + ":" + pwd.getText().toString()
+ "\n");
        fw.close();

        Toast.makeText(this, "3rd party credentials saved successfully!",
Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        Toast.makeText(this, "File error occurred",
Toast.LENGTH_SHORT).show();
        Log.d("Diva", "File error: " + e.getMessage());
    }
}
```

donde:

- **File sdir = Environment.getExternalStorageDirectory();**: Esta línea obtiene el directorio del almacenamiento externo del dispositivo para poder guardar archivos en esa ubicación.
- **File uinfo = new File(sdir.getAbsolutePath() + "/.uinfo.txt");**: Esta línea crea una referencia al fichero **.uinfo.txt** dentro del almacenamiento externo del dispositivo. El punto inicial del nombre: **.uinfo.txt**, hace que el fichero sea “oculto” en sistemas tipo Unix/Linux, pero no lo protege.
- **fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n");**: Esta línea escribe el usuario y la contraseña en texto claro dentro del fichero, separados por dos puntos.

El fallo principal es que la app guarda credenciales sensibles en un fichero externo sin cifrado. Android recomienda usar almacenamiento interno para datos que no deban ser accesibles por otras apps, y distingue entre almacenamiento interno, almacenamiento externo específico de app y almacenamiento compartido.

Dónde se guarda realmente: En un emulador o dispositivo Android, el almacenamiento externo suele estar montado en la: **/sdcard/**, o **/storage/emulated/0/**. En nuestro caso el fichero está en: **/sdcard/.uinfo.txt**.

Comprobamos en el dispositivo android:

```
(xxniwexx@kali)-[~]  
$ adb shell ls -la /sdcard/ | grep uinfo  
-rw-rw----  1 root sdcard_rw   15 2026-05-06 03:57 .uinfo.txt
```

```
(xxniwexx@kali)-[~]  
$ adb shell cat /sdcard/.uinfo.txt  
alumno6:pass6
```

La causa raíz es usar el almacenamiento externo para guardar información sensible sin cifrado. Aunque el fichero tenga un nombre oculto, esto no proporciona seguridad. La mitigación consiste en no almacenar contraseñas en el dispositivo y, si es imprescindible persistir información sensible, utilizar almacenamiento interno cifrado y claves gestionadas mediante Android Keystore.

Tabla resumen del Reto 6:

Campo	Resultado
Reto	Insecure Data Storage - Part 4
Clase vulnerable	<code>InsecureDataStorage4Activity.java</code>
Método vulnerable	<code>saveCredentials(View view)</code>
Campos leídos	<code>R.id.ids4Usr</code> , <code>R.id.ids4Pwd</code>
API problemática	<code>Environment.getExternalStorageDirectory()</code> , <code>FileWriter</code>
Directorio usado	Almacenamiento externo
Ruta habitual	<code>/sdcard/.uinfo.txt</code> o <code>/storage/emulated/0/.uinfo.txt</code>
Nombre del fichero	<code>.uinfo.txt</code>
Formato del contenido	<code>usuario:contraseña</code>
Tipo de fallo	Almacenamiento inseguro de credenciales
Causa raíz	Usuario y contraseña escritos en texto claro en almacenamiento externo
Fragmento vulnerable	<code>fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n")</code>
Cómo se comprueba	Leyendo <code>/sdcard/.uinfo.txt</code> con <code>adb shell</code>
Impacto	Exposición de credenciales a través del almacenamiento externo
Mitigación	No guardar credenciales en almacenamiento externo. Usar almacenamiento interno cifrado si es imprescindible

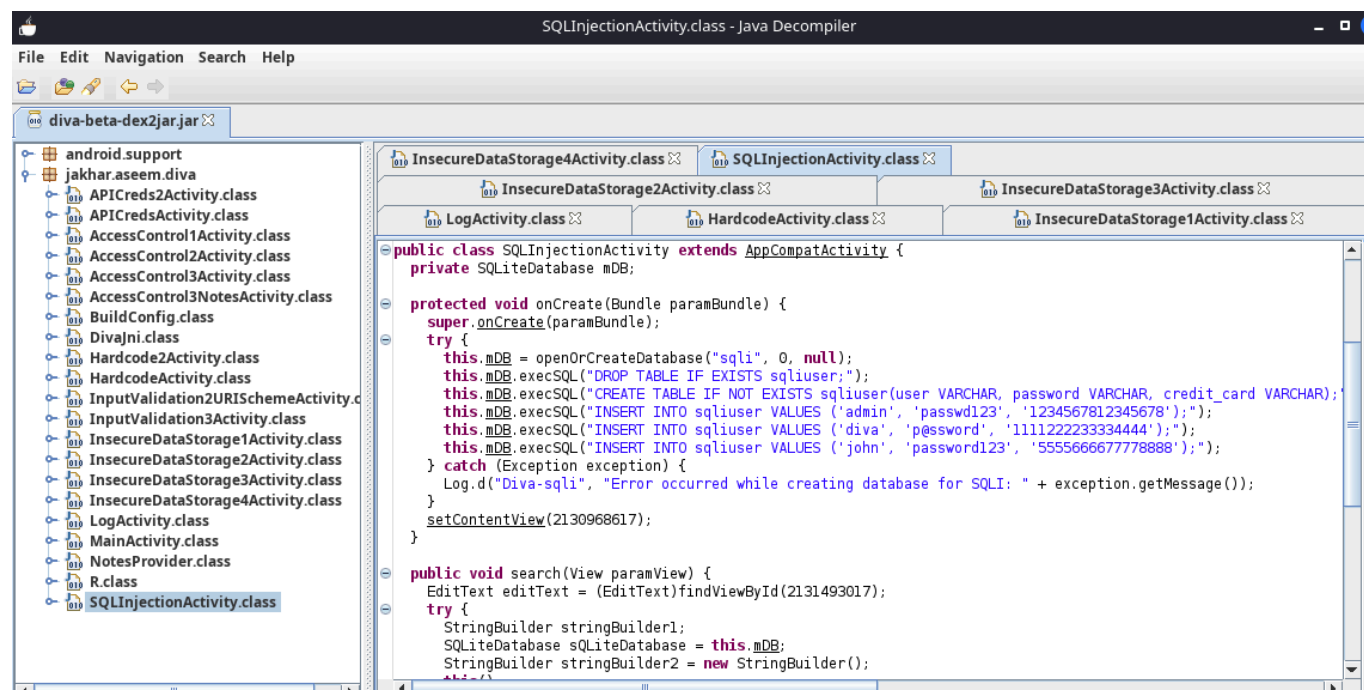
2.7 Input Validation Issues — Part 1

Como ya se analizó y resolvió en el [Reto 7 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: La falta de validación de entrada que permite modificar la consulta SQL ejecutada por la aplicación. En este caso, la app tiene una base de datos SQLite local con una tabla llamada: `sqluser`, que contiene:

```
usuario
contraseña
tarjeta de crédito
```

El reto consiste en conseguir que la consulta devuelva más información de la que debería.

Dentro del paquete `jakhar.aseem.diva`, vamos a localizar la clase `SQLInjectionActivity`, asociada a este reto:



Métodos relevantes: En esta clase hay dos métodos importantes:

- `onCreate(Bundle savedInstanceState)`: Crea la base `sqli`, la tabla `sqliuser` y añade usuarios como `admin`, `diva` y `john`

```
mDB = openOrCreateDatabase("sqli", MODE_PRIVATE, null);
mDB.execSQL("DROP TABLE IF EXISTS sqliuser;");
mDB.execSQL("CREATE TABLE IF NOT EXISTS sqliuser(user VARCHAR,
password VARCHAR, credit_card VARCHAR);");

mDB.execSQL("INSERT INTO sqliuser VALUES ('admin', 'passwd123',
'1234567812345678');");
mDB.execSQL("INSERT INTO sqliuser VALUES ('diva', 'p@ssword',
'1111222233334444');");
mDB.execSQL("INSERT INTO sqliuser VALUES ('john', 'password123',
'5555666677778888');");
```

- `search(View view)`: Este método `search()` Recoge la entrada del usuario y ejecuta una consulta con `rawQuery()` que concatena la entrada del campo `ivi1search`.

```
EditText srchtxt = (EditText) findViewById(R.id.ivi1search);

cr = mDB.rawQuery(
    "SELECT * FROM sqliuser WHERE user = '"
    + srchtxt.getText().toString()
    + "'",
    null
);
```

El código problemático es:

```
"SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText().toString() +
"""
```

Como el texto se concatena directamente dentro de la consulta SQL, el usuario puede introducir una cadena que altere la lógica del `WHERE`, como por ejemplo:

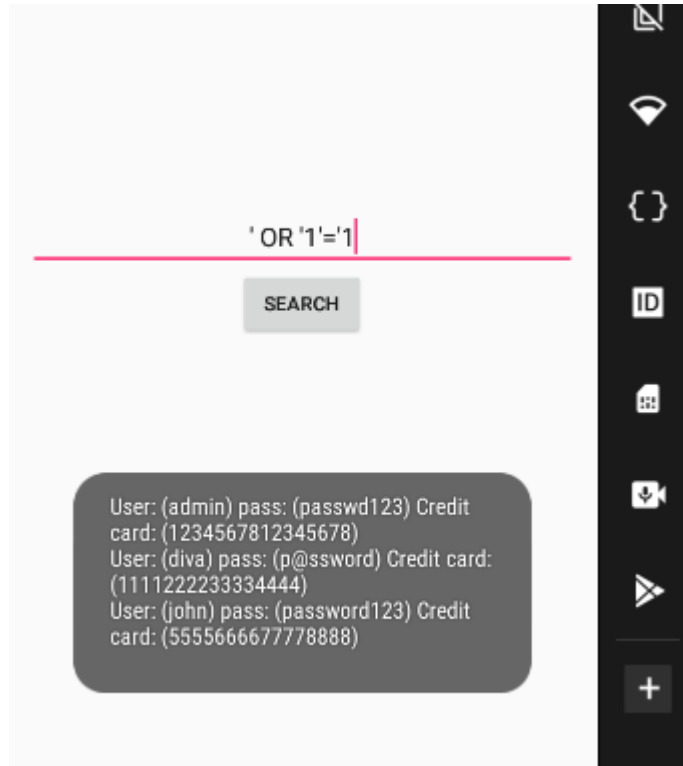
```
' OR '1'='1
```

La consulta resultante sería:

```
SELECT * FROM sqliuser WHERE user = '' OR '1'='1'
```

Como '1'='1' siempre es verdadero, la consulta devuelve todos los registros.

Comprobamos en el dispositivo android:



La causa raíz es tratar la entrada del usuario como parte de la sentencia SQL. Esto permite introducir payloads como

```
' OR '1'='1'
```

consiguiendo que la condición del **WHERE** sea siempre verdadera y que la consulta devuelva todos los registros de la tabla, incluyendo usuarios, contraseñas y tarjetas de crédito. La mitigación consiste en usar consultas parametrizadas mediante **?** y pasar los valores del usuario como argumentos separados, evitando que sean interpretados como código SQL.

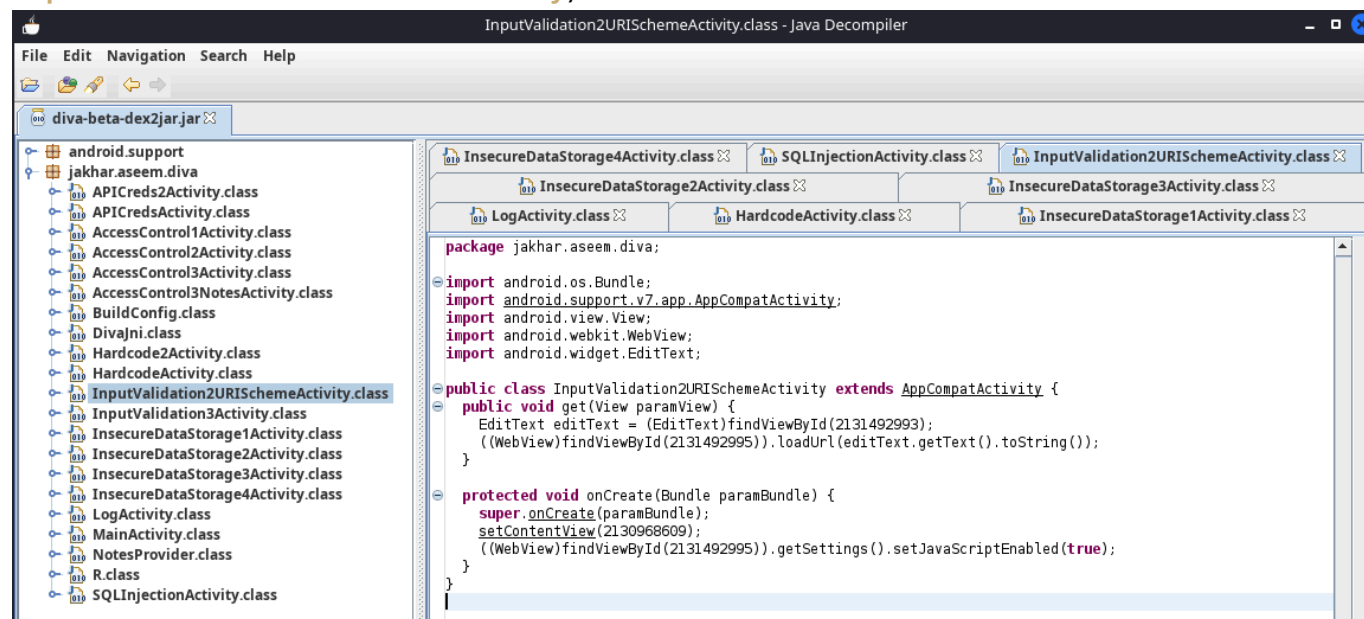
Tabla resumen del Reto 7:

Campo	Resultado
Reto	Input Validation Issues - Part 1
Clase vulnerable	<code>SQLInjectionActivity.java</code>
Métodos relevantes	<code>onCreate()</code> , <code>search(View view)</code>
Campo leído	<code>R.id.iv1search</code>
API problemática	<code>SQLiteDatabase.rawQuery()</code>
Base de datos	<code>sqli</code>
Tabla	<code>sqliuser</code>
Columnas	<code>user</code> , <code>password</code> , <code>credit_card</code>
Tipo de fallo	SQL Injection
Causa raíz	Concatenación directa de entrada del usuario en una consulta SQL
Fragmento vulnerable	<code>"SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText().toString() + '"</code>
Payload de prueba	<code>' OR '1'='1</code>
Resultado	Se muestran todos los registros de la tabla
Impacto	Exposición de usuarios, contraseñas y tarjetas de crédito
Mitigación	Usar consultas parametrizadas con <code>?</code> y argumentos separados

2.8 Input Validation Issues — Part 2

Como ya analizamos el [Reto 8 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Confiar en una URI controlada por el usuario. Identificaremos una falta de validación de entrada en una funcionalidad que carga contenido en un WebView. La app espera que el usuario introduzca una dirección web, pero no valida si la URI empieza por `http://`, `https://`, `file://` u otro esquema peligroso. Esto permite cargar recursos locales del dispositivo, por ejemplo: `file:///sdcard/.uinfo.txt`. En Android, este tipo de fallo se considera **Unsafe URI Loading**, que ocurre cuando una aplicación no valida correctamente una **URI** antes de cargarla dentro de un WebView.

Dentro del paquete `jakhar.aseem.diva`, se localiza la clase **`InputValidation2URISchemeActivity`**, asociada a este reto:



Métodos relevantes: En esta clase hay dos métodos importantes:

- `onCreate(Bundle savedInstanceState)`: Configura el WebView.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_input_validation2_urischeme);

    WebView wview = (WebView) findViewById(R.id.ivi2wview);
    WebSettings wset = wview.getSettings();
    wset.setJavaScriptEnabled(true);
}
```

- `get(View view)`: Carga la URI introducida por el usuario.

```
public void get(View view) {  
    EditText uriText = (EditText) findViewById(R.id.ivi2uri);  
    WebView wview = (WebView) findViewById(R.id.ivi2wview);  
    wview.loadUrl(uriText.getText().toString());  
}
```

El código oficial de DIVA muestra que la clase `InputValidation2URISchemeActivity` obtiene el texto del campo `ivi2uri` y lo pasa directamente a `wview.loadUrl(...)` sin validación previa.

`uriText.getText().toString()`: Obtiene el texto introducido por el usuario, y `wview.loadUrl(...)`: Lo pasa directamente al WebView.

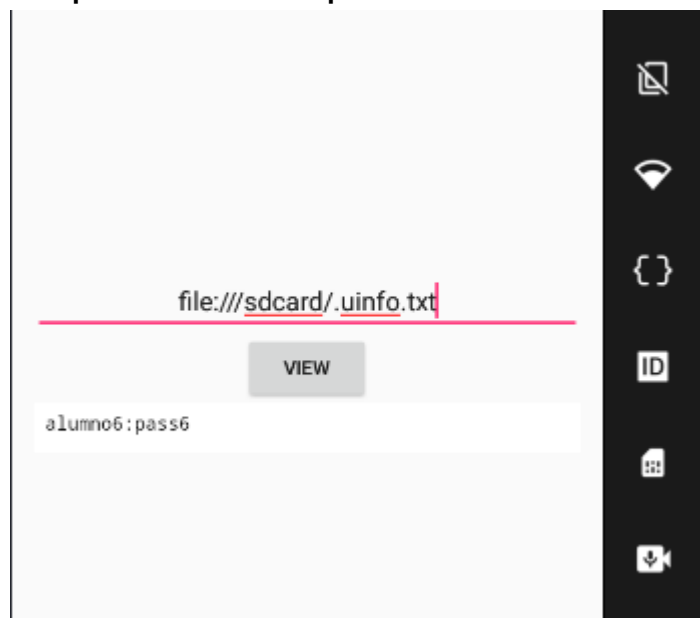
No comprueba:

- Si la URI pertenece a un dominio permitido.
- Si el esquema es seguro.

No bloquea esquemas como:

- `file:///`.
- `content:///`.
- `javascript:.`

Comprobamos en el dispositivo android:



La causa raíz es confiar en una URI controlada por el usuario. Esto permite introducir valores como `file:///sdcard/.uinfo.txt`, haciendo que el WebView cargue un fichero local del dispositivo. En combinación con el reto anterior, donde se almacenan credenciales en `/sdcard/.uinfo.txt`, esta vulnerabilidad permite visualizar esas credenciales desde la propia aplicación. La mitigación consiste en validar estrictamente la **URI** antes de cargarla, permitir únicamente esquemas y dominios autorizados, bloquear esquemas peligrosos como `file:///` y deshabilitar JavaScript si no es necesario.

Tabla resumen del Reto 8:

Campo	Resultado
Reto	Input Validation Issues - Part 2
Clase vulnerable	<code>InputValidation2URISchemeActivity.java</code>
Métodos relevantes	<code>onCreate()</code> , <code>get(View view)</code>
Campo leído	<code>R.id.iv2uri</code>
Componente afectado	<code>WebView</code>
API problemática	<code>WebView.loadUrl()</code>
Configuración relevante	<code>wset.setJavaScriptEnabled(true)</code>
Tipo de fallo	Unsafe URI Loading / validación insuficiente de entrada
Causa raíz	Se carga una URI controlada por el usuario sin validar esquema, dominio ni ruta
Fragmento vulnerable	<code>wview.loadUrl(uriText.getText().toString())</code>
Payload de prueba	<code>file:///sdcard/.uinfo.txt</code>
Resultado	El WebView puede mostrar el contenido de un fichero local
Impacto	Exposición de ficheros locales o carga de contenido no confiable
Mitigación	Validar URI con allowlist. Permitir solo <code>https</code> . Bloquear <code>file://</code> . Deshabilitar JavaScript si no es necesario

2.9: Access Control Issues — Part 1

Como ya se analizó y resolvió en el [Reto 9 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Exponer una Activity sensible mediante un **intent-filter** sin aplicar controles de acceso. Identificaremos que una pantalla sensible de la aplicación puede abrirse sin autenticación ni control de permisos.

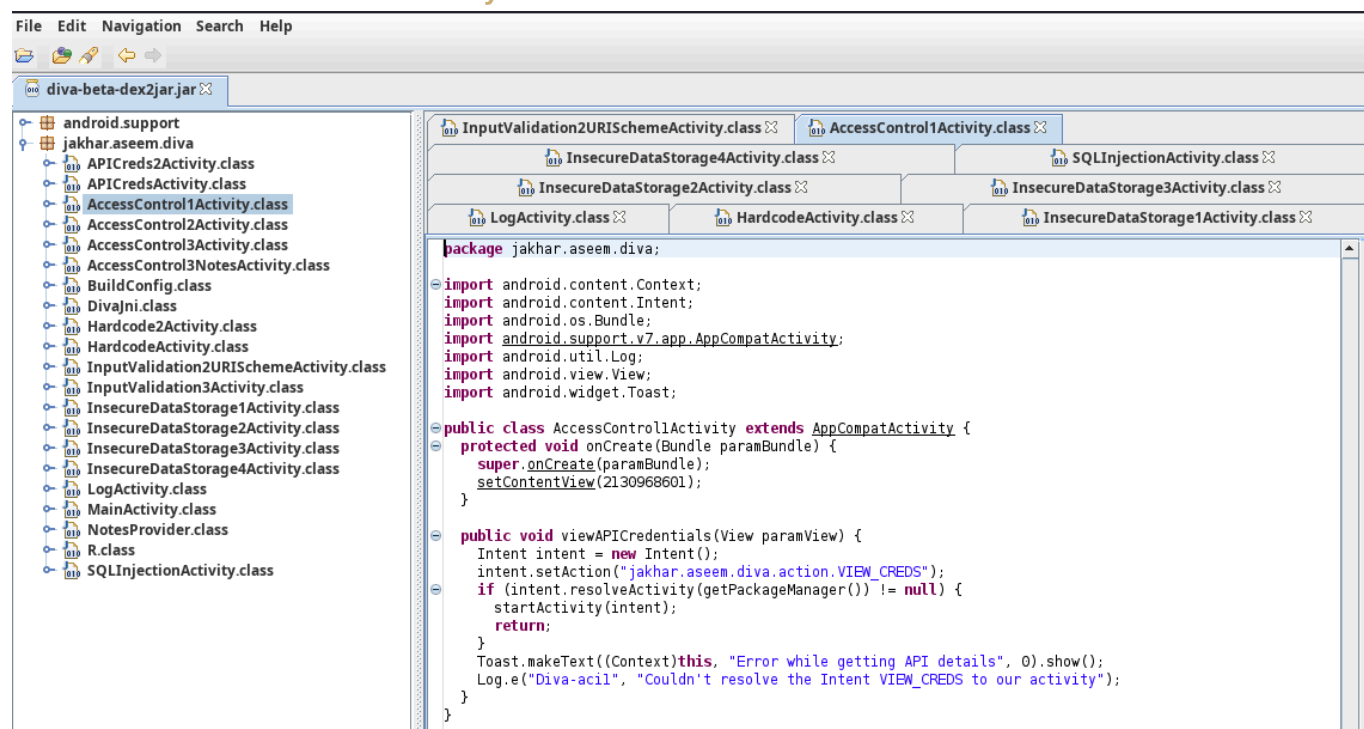
En este caso, la pantalla vulnerable muestra:

```
API Key
API User name
API Password
```

El problema es que esa pantalla puede invocarse directamente mediante una acción de Intent.

Dentro del paquete **jakhar.aseem.diva**, se localizan las clases **AccessControl1Activity** y **APICredsActivity**: Además, se analiza el **AndroidManifest.xml**, donde se declara un **intent-filter** que permite lanzar **APICredsActivity** mediante una acción personalizada.

La clase **AccessControl1Activity**:



```

public void viewAPICredentials(View view) {
    Intent i = new Intent();
    i.setAction("jakhar.aseem.diva.action.VIEW_CREDS");

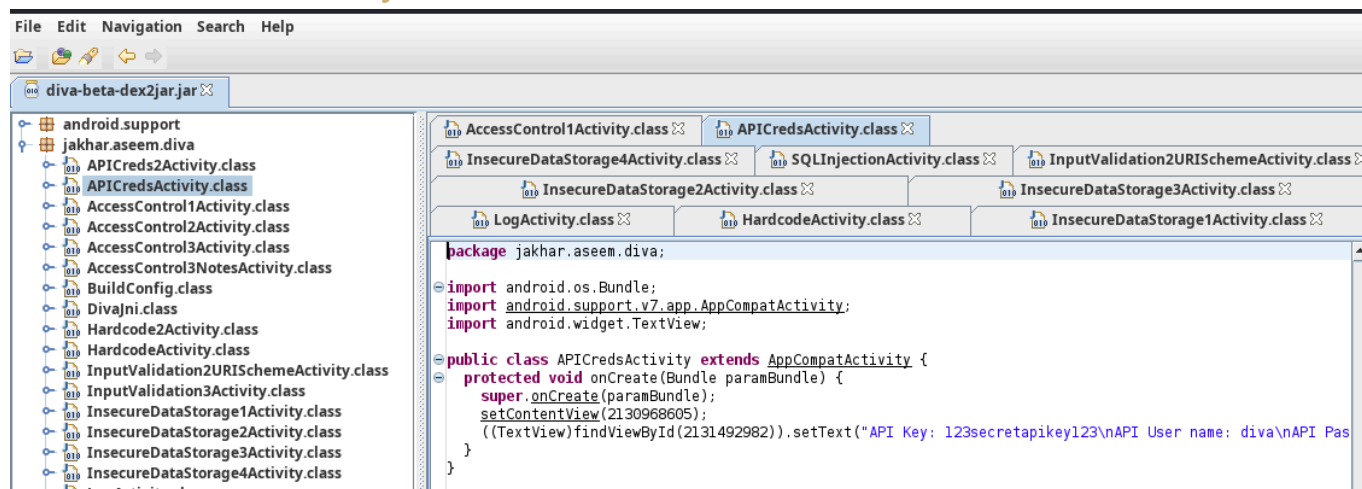
    if (i.resolveActivity(getPackageManager()) != null) {
        startActivity(i);
    } else {
        Toast.makeText(this, "Error while getting API details",
        Toast.LENGTH_SHORT).show();
        Log.e("Diva-aci1", "Couldn't resolve the Intent VIEW_CREDS to our
        activity");
    }
}
}

```

donde:

- Observamos que `viewAPICredentials()` crea un Intent.
- Le asigna la acción `jakhar.aseem.diva.action.VIEW_CREDS`.
- Después llama a `startActivity(i)`.

La clase `APICredsActivity`:



```

public class APICredsActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_apicreds);

        TextView apicview = (TextView) findViewById(R.id.apicTextView);

        String apidetails = "API Key: 123secretapikey123\n"
            + "API User name: diva\n"
            + "API Password: p@ssword";

        apicview.setText(apidetails);
    }
}

```

```
}  
}
```

donde:

- La clase `APICredsActivity` es la pantalla que muestra las credenciales.
- La propia Activity contiene las credenciales en una cadena y las muestra directamente en un `TextView`: `API Key: 123secretapikey123, API User name: diva y API Password: p@ssword`.

Declaración en `AndroidManifest.xml`: En el `AndroidManifest.xml`, `APICredsActivity` aparece declarada con un `intent-filter`:

```
<activity  
    android:name=".APICredsActivity"  
    android:label="@string/apic_label">  
  
    <intent-filter>  
        <action android:name="jakhar.aseem.diva.action.VIEW_CREDS" />  
        <category android:name="android.intent.category.DEFAULT" />  
    </intent-filter>  
  
</activity>
```

Esto significa que Android puede resolver esa Activity cuando alguien lance un `Intent` con la acción: `jakhar.aseem.diva.action.VIEW_CREDS`.

En el manifiesto de DIVA, `APICredsActivity` está declarada con ese `intent-filter`, incluyendo la acción `jakhar.aseem.diva.action.VIEW_CREDS` y la categoría `android.intent.category.DEFAULT`.

Explicación del fallo: El patrón vulnerable es: `Activity sensible` → `intent-filter público` → `sin permisos` → `acceso externo`. La aplicación hace lo siguiente:

- Define una pantalla sensible: `APICredsActivity`.
- Esa pantalla muestra credenciales de API.
- En el manifiesto, la pantalla tiene un `intent-filter`.
- Ese `intent-filter` permite abrirla mediante una acción personalizada.
- No hay comprobación de autenticación.
- No hay permiso personalizado.
- No hay validación de quién lanza el `Intent`.

El problema no es usar `Intent`, sino exponer una pantalla sensible sin control de acceso.

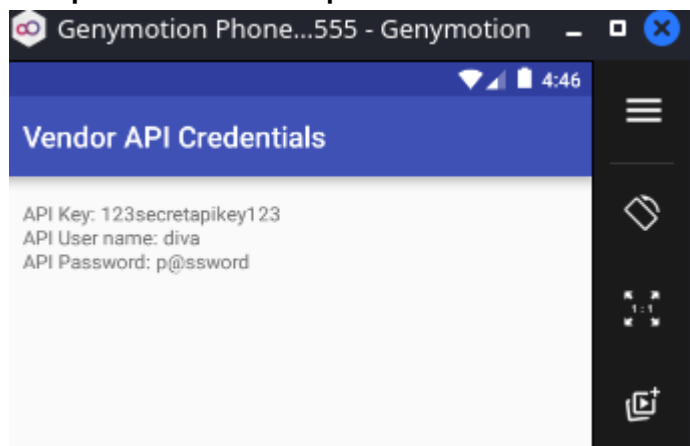
Por qué es vulnerable: Una Activity con un `intent-filter` puede ser invocada mediante un `Intent` implícito. En este caso, como la acción es conocida:

```
jakhar.aseem.diva.action.VIEW_CREDS
```

Se puede lanzar directamente desde ADB o desde otra aplicación.

Eso permite saltarse el flujo normal de la app y abrir directamente la pantalla que muestra las credenciales.

Comprobamos en el dispositivo android:



La causa raíz es exponer una Activity sensible mediante un `intent-filter` sin aplicar controles de acceso. La mitigación consiste en no exportar componentes sensibles, eliminar `intent-filter` innecesarios, proteger la actividad mediante permisos personalizados y validar la autorización antes de mostrar información confidencial.

Tabla resumen del Reto 9:

Campo	Resultado
Reto	Access Control Issues - Part 1
Clase iniciadora	<code>AccessControl1Activity.java</code>
Clase sensible	<code>APICredsActivity.java</code>
Fichero clave	<code>AndroidManifest.xml</code>
Método relevante	<code>viewAPICredentials(View view)</code>
Componente afectado	<code>Activity</code>
Acción del Intent	<code>jakhar.aseem.diva.action.VIEW_CREDS</code>
API problemática	<code>startActivity()</code> con <code>Intent</code> implícito
Datos expuestos	API Key, API User name, API Password
Tipo de fallo	Control de acceso insuficiente
Causa raíz	<code>Activity</code> sensible expuesta mediante <code>intent-filter</code> sin permisos ni autenticación
Fragmento vulnerable	<code><action android:name="jakhar.aseem.diva.action.VIEW_CREDS" /></code>
Comando de prueba	<code>adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS</code>
Resultado	Se abre directamente la pantalla de credenciales
Impacto	Exposición de credenciales a apps externas o usuarios con ADB
Mitigación	<code>android:exported="false"</code> . Eliminar <code>intent-filter</code> . Usar permisos personalizados y validar autorización

2.10 Access Control Issues — Part 2

Como ya se analizó y resolvió en el [Reto 10 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Delegar una decisión de seguridad en un parámetro controlable por el caller externo. Accederemos a las credenciales de la API de **TVEETER** sin conocer el PIN. La app plantea un flujo aparentemente legítimo:

```
Registrarse en Tveeter
Recibir un PIN
Introducir el PIN
Ver credenciales de API
```

Pero el fallo está en que la app decide si debe comprobar el PIN usando un valor recibido por Intent.

Tenemos que revisar:

- Clases vulnerables dentro del paquete `jakhar.aseem.diva`:
 - `AccessControl2Activity`: Recoge el estado de un `RadioButton` y lo mete como extra en un Intent.
 - `APICreds2Activity`: Muestra o no las credenciales según un extra booleano.
- `AndroidManifest.xml`: Declara `APICreds2Activity` con un `intent-filter` accesible por la acción `jakhar.aseem.diva.action.VIEW_CREDS2`.
- `strings.xml`: En `strings.xml`, el recurso `R.string.chk_pin` corresponde al texto: `check_pin`.

La clase `AccessControl2Activity`:

```
public void viewAPICredentials(View view) {
    RadioButton rbregnow = (RadioButton) findViewById(R.id.aci2rbregnow);

    Intent i = new Intent();
    boolean chk_pin = rbregnow.isChecked();

    i.setAction("jakhar.aseem.diva.action.VIEW_CREDS2");
    i.putExtra(getString(R.string.chk_pin), chk_pin);

    if (i.resolveActivity(getPackageManager()) != null) {
        startActivity(i);
    } else {
        Toast.makeText(this, "Error while getting Tveeter API details",
            Toast.LENGTH_SHORT).show();
        Log.e("Diva-aci1", "Couldn't resolve the Intent VIEW_CREDS2 to our
            activity");
    }
}
```

donde:

- Vemos que la acción usada es `jakhar.aseem.diva.action.VIEW_CREDS2` y que el valor booleano `chk_pin` se pasa como extra al `Intent`.
- `i.putExtra(getString(R.string.chk_pin), chk_pin);` Equivale conceptualmente a: `i.putExtra("check_pin", chk_pin);`. Ya que el fichero `strings.xml` contiene el valor `check_pin`, que es la clave usada para recuperar el extra booleano en la siguiente `Activity`.

La clase **APICreds2Activity**:

```
if (bcheck == false) {
    String apidetails = "TVEETER API Key: secrettveeterapikey\n"
        + "API User name: diva2\n"
        + "API Password: p@ssword2";

    apicview.setText(apidetails);
} else {
    apicview.setText("Register yourself at http://payatu.com to get your PIN and then login with that PIN!");
    pintext.setVisibility(View.VISIBLE);
    vbutton.setVisibility(View.VISIBLE);
}
```

donde:

- El código oficial de **APICreds2Activity** muestra que si `bcheck == false`, la app muestra directamente las credenciales `TVEETER API Key`, `API User name` y `API Password`.
- Si `bcheck` es `true`, muestra el formulario para introducir el PIN.

AndroidManifest.xml: En el `AndroidManifest.xml`, **APICreds2Activity** aparece declarada con un `intent-filter`:

```
<activity
    android:name=".APICreds2Activity"
    android:label="@string/apic2_label">

    <intent-filter>
        <action android:name="jakhar.aseem.diva.action.VIEW_CREDS2" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>

</activity>
```

Esto significa que la `Activity` puede ser abierta mediante un `Intent` con la acción: `jakhar.aseem.diva.action.VIEW_CREDS2`.

El manifiesto de DIVA declara **APICreds2Activity** con ese `intent-filter`, incluyendo la acción `jakhar.aseem.diva.action.VIEW_CREDS2` y la categoría `android.intent.category.DEFAULT`.

Explicación de la vulnerabilidad: El patrón vulnerable es:

Activity sensible → intent-filter público → extra manipulable → bypass del PIN

La lógica vulnerable es esta:

```
boolean bcheck = i.getBooleanExtra(getString(R.string.chk_pin), true);

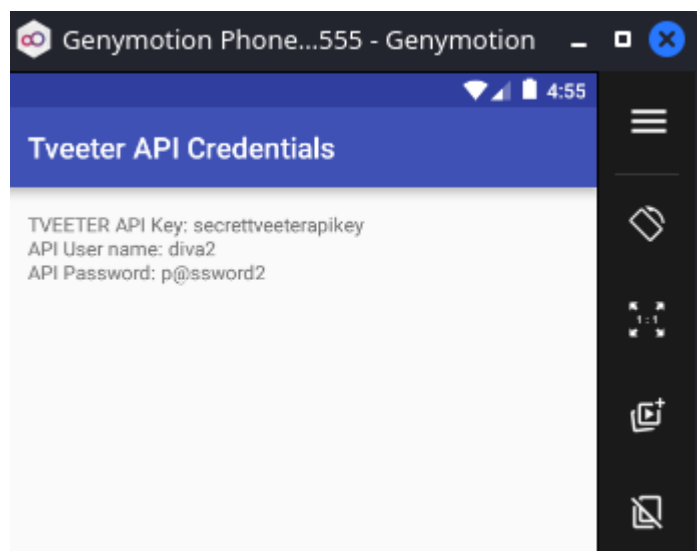
if (bcheck == false) {
    // mostrar credenciales
} else {
    // pedir PIN
}
```

donde:

- El problema es que **bcheck** viene del **Intent**.
- Si una aplicación externa, Drozer o ADB lanza la Activity y envía: **check_pin = false**, entonces la app interpreta que no debe comprobar el **PIN** y muestra directamente las credenciales.

Comprobamos en el dispositivo android:

```
dz> run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICreds2Activity --extra boolean check_pin false
Attempting to run shell module
dz> 
```



La causa raíz es delegar una decisión de seguridad en un parámetro controlable por el caller externo. La mitigación consiste en no confiar en extras de **Intent** para autorización, no exportar actividades sensibles, proteger componentes con permisos personalizados y validar el PIN o la sesión dentro del propio componente sensible antes de mostrar información confidencial.

Tabla resumen del Reto 10:

Campo	Resultado
Reto	Access Control Issues - Part 2
Clase iniciadora	<code>AccessControl2Activity.java</code>
Clase sensible	<code>APICreds2Activity.java</code>
Fichero clave	<code>AndroidManifest.xml</code>
Recurso relevante	<code>R.string.chk_pin = check_pin</code>
Método vulnerable	<code>APICreds2Activity.onCreate()</code>
Componente afectado	<code>Activity</code>
Acción del Intent	<code>jakhar.aseem.diva.action.VIEW_CREDS2</code>
Extra manipulable	<code>check_pin</code>
Valor usado para el bypass	<code>false</code>
Datos expuestos	TVEETER API Key, API User name, API Password
Tipo de fallo	Control de acceso insuficiente / confianza en datos externos
Causa raíz	La app decide si pedir PIN usando un extra del <code>Intent</code>
Fragmento vulnerable	<code>boolean bcheck = i.getBooleanExtra(getString(R.string.chk_pin), true)</code>
Comando de prueba	<code>adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS2 --ez check_pin false</code>
Resultado	Se muestran las credenciales sin introducir PIN
Impacto	Bypass de PIN y exposición de credenciales
Mitigación	No confiar en extras para autorización. <code>android:exported="false"</code> . Permisos personalizados. Validación interna del PIN

2.11 Access Control Issues — Part 3

Como ya se analizó y resolvió en el [Reto 11 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: No aplicar un control en el componente que realmente expone los datos. Accederemos a las notas privadas de la aplicación sin conocer el PIN configurado por el usuario. La app aparenta proteger las notas con un PIN, pero el problema es que esas notas están almacenadas en un `ContentProvider` accesible desde fuera de la aplicación.

Clases y ficheros relevantes:

```
AccessControl3Activity
AccessControl3NotesActivity
NotesProvider
AndroidManifest.xml
```

donde:

- La clase `AccessControl3Activity` permite crear un PIN y después abrir la pantalla de notas.
- La clase `AccessControl3NotesActivity` comprueba el PIN antes de mostrar las notas desde la interfaz. Sin embargo, las notas están realmente expuestas por `NotesProvider`, declarado en el `AndroidManifest.xml` con `android:exported="true"`.

El código en `AccessControl3Activity`: En `AccessControl3Activity`, la app guarda el PIN en `SharedPreferences`:

```
SharedPreferences spref =
PreferenceManager.getDefaultSharedPreferences(this);
SharedPreferences.Editor spedit = spref.edit();

EditText pinTxt = (EditText) findViewById(R.id.aci3Pin);
String pin = pinTxt.getText().toString();

spedit.putString(getString(R.string.pkey), pin);
spedit.commit();

Después, cuando el usuario pulsa el botón para ver las notas, se abre la
pantalla AccessControl3NotesActivity:

public void goToNotes(View view) {
    Intent i = new Intent(this, AccessControl3NotesActivity.class);
    startActivity(i);
}
```

donde:

- El código oficial muestra que `AccessControl3Activity` guarda el PIN en preferencias y lanza `AccessControl3NotesActivity` mediante un `Intent` explícito.

EL código en `AccessControl3NotesActivity`: Esta clase sí comprueba el PIN antes de mostrar las notas en la interfaz:

```

SharedPreferences spref =
PreferenceManager.getDefaultSharedPreferences(this);
String pin = spref.getString(getString(R.string.pkey), "");
String userpin = pinTxt.getText().toString();

if (userpin.equals(pin)) {
    Cursor cr = getContentResolver().query(
        NotesProvider.CONTENT_URI,
        new String[] {"_id", "title", "note"},
        null,
        null,
        null
    );
}

```

donde:

- A primera vista parece correcto porque la app pide el PIN antes de consultar las notas.
- Pero el fallo es que esa protección sólo está en la interfaz. El `ContentProvider` que almacena las notas puede consultarse directamente desde fuera.

Código en `NotesProvider`: La clase vulnerable principal es: `NotesProvider`. En ella se define:

- La base de datos:

```

static final String DBNAME = "divanotes.db";
static final String TABLE = "notes";
static final String C_ID = "_id";
static final String C_TITLE = "title";
static final String C_NOTE = "note";

```

- La autoridad del proveedor:

```

static final String AUTHORITY = "jakhar.aseem.diva.provider.notesprovider";

```

- La URI pública:

```

static final Uri CONTENT_URI =
    Uri.parse("content://" + AUTHORITY + "/" + TABLE);

```

Por tanto, la URI final para consultar las notas es:

`content://jakhar.aseem.diva.provider.notesprovider/notes.`

El código de NotesProvider define la base `divanotes.db`, la tabla `notes`, la autoridad `jakhar.aseem.diva.provider.notesprovider` y la `CONTENT_URI` construida como `content://jakhar.aseem.diva.provider.notesprovider/notes`.

Datos que contiene el proveedor: En `NotesProvider`, el método `onCreate()` del `helper` de base de datos inserta varias notas de prueba, por ejemplo:

```
office
home
holiday
Expense
Exercise
Weekend
```

donde:

- Algunas notas contienen información privada, como reuniones, llamadas, compras, gastos o planes personales.
- El proveedor inserta estas notas en la tabla `notes` al crear la base de datos.

Declaración vulnerable en `AndroidManifest.xml`: El problema principal aparece en el `AndroidManifest.xml`:

```
<provider
    android:name=".NotesProvider"
    android:authorities="jakhar.aseem.diva.provider.notesprovider"
    android:enabled="true"
    android:exported="true" >
</provider>
```

donde:

- La línea vulnerable es: `android:exported="true"`.
- Además, no aparece ningún permiso de lectura o escritura, como:
 - `android:readPermission="..."`.
 - `android:writePermission="..."`

El manifiesto oficial declara `.NotesProvider` con la autoridad `jakhar.aseem.diva.provider.notesprovider`, `android:enabled="true"` y `android:exported="true"`.

Explicación del fallo: El patrón vulnerable es: **Notas privadas** → **ContentProvider exportado** → **sin permisos** → **acceso externo**. La aplicación intenta proteger las notas con un PIN en la pantalla **AccessControl3NotesActivity**. Pero esa protección está en este nivel: **Interfaz de usuario / Activity**. Mientras que los datos reales están en este otro nivel: **ContentProvider / base de datos SQLite**. El problema es que el **ContentProvider** está exportado y no exige permisos. Por tanto, otra app o una herramienta como adb/Drozer puede consultar directamente la URI del proveedor sin pasar por la pantalla que pide el PIN.

Un **ContentProvider** es un componente Android diseñado para exponer datos a otras aplicaciones. Si se exporta sin controles, cualquier otra app puede intentar acceder a sus datos. En este reto: **android:exported="true"** hace que el proveedor sea accesible desde fuera.

Comprobamos en el dispositivo android:

```
dz> run app.provider.info -a jakhar.aseem.diva
Attempting to run shell module
Package: jakhar.aseem.diva
  Authority: jakhar.aseem.diva.provider.notesprovider
    Read Permission: null
    Write Permission: null
    Content Provider: jakhar.aseem.diva.NotesProvider
    Multiprocess Allowed: False
    Grant Uri Permissions: False
dz> █
```

```
dz> run scanner.provider.finduris -a jakhar.aseem.diva
Attempting to run shell module
Scanning jakhar.aseem.diva...
Got a response from content Uri: content://jakhar.aseem.diva.provider.notesprovider/notes/
Got a response from content Uri: content://jakhar.aseem.diva.provider.notesprovider/notes
No response from content URI: content://jakhar.aseem.diva.provider.notesprovider
No response from content URI: content://jakhar.aseem.diva.provider.notesprovider/

For sure accessible content URIs:
  content://jakhar.aseem.diva.provider.notesprovider/notes/
  content://jakhar.aseem.diva.provider.notesprovider/notes
dz> █
```

```
dz> run app.provider.query content://jakhar.aseem.diva.provider.notesprovider/notes/
Attempting to run shell module
| _id | title      | note                                     |
| 5   | Exercise   | Alternate days running                  |
| 4   | Expense    | Spent too much on home theater         |
| 6   | Weekend    | b3333333333333r                        |
| 3   | holiday    | Either Goa or Amsterdam                |
| 2   | home       | Buy toys for baby, Order dinner       |
| 1   | office     | 10 Meetings. 5 Calls. Lunch with CEO  |
dz> █
```

La causa raíz es aplicar el control de acceso en la Activity, pero no en el componente que realmente expone los datos. La mitigación consiste en marcar el provider como **android:exported="false"** si no debe ser accesible externamente, o protegerlo con permisos personalizados y validaciones internas en sus métodos **query()**, **insert()**, **update()** y **delete()**.

Tabla resumen del Reto 11:

Campo	Resultado
Reto	Access Control Issues - Part 3
Clase de configuración	<code>AccessControl3Activity.java</code>
Clase de visualización	<code>AccessControl3NotesActivity.java</code>
Clase vulnerable principal	<code>NotesProvider.java</code>
Fichero clave	<code>AndroidManifest.xml</code>
Componente afectado	<code>ContentProvider</code>
Authority	<code>jakhar.aseem.diva.provider.notesprovider</code>
URI vulnerable	<code>content://jakhar.aseem.diva.provider.notesprovider/notes</code>
Base de datos	<code>divanotes.db</code>
Tabla	<code>notes</code>
Columnas	<code>_id, title, note</code>
Declaración problemática	<code>android:exported="true"</code>
Tipo de fallo	Control de acceso insuficiente
Causa raíz	Provider exportado sin permisos; el PIN solo protege la interfaz, no el acceso real a datos
Fragmento vulnerable	<code><provider android:name=".NotesProvider" android:authorities="jakhar.aseem.diva.provider.notesprovider" android:exported="true" /></code>
Comando de prueba	<code>adb shell content query --uri content://jakhar.aseem.diva.provider.notesprovider/notes</code>
Resultado	Se muestran las notas privadas sin introducir PIN
Impacto	Exposición de información privada almacenada en el provider
Mitigación	Usar <code>android:exported="false"</code> , permisos personalizados y controles de autorización dentro del provider

2.12 Hardcoding Issues — Part 2

Como ya se analizó y resolvió en el [Reto 12 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Confiar en un secreto almacenado en el cliente. Encontraremos una clave de acceso hardcodeada, pero esta vez no está directamente en el código Java de la actividad. La aplicación usa una clase Java llamada: `DivaJni` que carga una librería nativa: `libdivajni.so`. Dentro de esa librería se encuentra la clave usada para validar el acceso.

Clases y ficheros relevantes:

- `Hardcode2Activity`.
- `DivaJni`.
- En el código fuente nativo:
 - `libdivajni.so`.
 - `divajni.c`.

El código oficial muestra que `Hardcode2Activity` crea un objeto `DivaJni` y llama a `djni.access(...)` con el texto introducido por el usuario.

Código en `Hardcode2Activity`: La actividad vulnerable es:

```
public class Hardcode2Activity extends AppCompatActivity {
    private DivaJni djni;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_hardcode2);
        djni = new DivaJni();
    }

    public void access(View view) {
        EditText hkey = (EditText) findViewById(R.id.hc2Key);

        if (djni.access(hkey.getText().toString()) != 0) {
            Toast.makeText(this, "Access granted! See you on the other side :)", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "Access denied! See you in hell :D", Toast.LENGTH_SHORT).show();
        }
    }
}
```

donde:

- `if (djni.access(hkey.getText().toString()) != 0)`: Aquí vemos que la validación no se hace directamente en Java, sino en el método nativo `access()` de la clase `DivaJni`.

Código en DivaJni.java: La clase `DivaJni` carga la librería nativa:

```
public class DivaJni {
    private static final String soName = "divajni";

    static {
        System.loadLibrary(soName);
    }

    public native int access(String key);
    public native int initiateLaunchSequence(String code);
}
```

donde:

- `System.loadLibrary(soName);`: Como `soName` vale: `divajni`, lo que hace android es cargar una librería llamada: `libdivajni.so`.

El código confirma que `DivaJni` define `soName = "divajni"`, carga esa librería mediante `System.loadLibrary(soName)` y declara los métodos nativos `access(String key)` e `initiateLaunchSequence(String code)`.

Código nativo vulnerable: En el código nativo `divajni.c` aparece la clave:

```
#define VENDORKEY "olsdfgad;lh"

Y el método nativo access() compara la entrada del usuario con esa clave:

JNIEXPORT jint JNICALL Java_jakhar_aseem_diva_DivaJni_access(
    JNIEnv * env,
    jobject obj,
    jstring jkey
) {
    const char * key = (*env)->GetStringUTFChars(env, jkey, 0);

    return ((strncmp(VENDORKEY, key, strlen(VENDORKEY))) ? 0 : 1);
}
```

donde:

- `#define VENDORKEY "olsdfgad;lh"`: El código fuente nativo define `VENDORKEY` con el valor `olsdfgad;lh` y `Java_jakhar_aseem_diva_DivaJni_access()` compara la entrada del usuario con ese valor usando `strncmp()`.

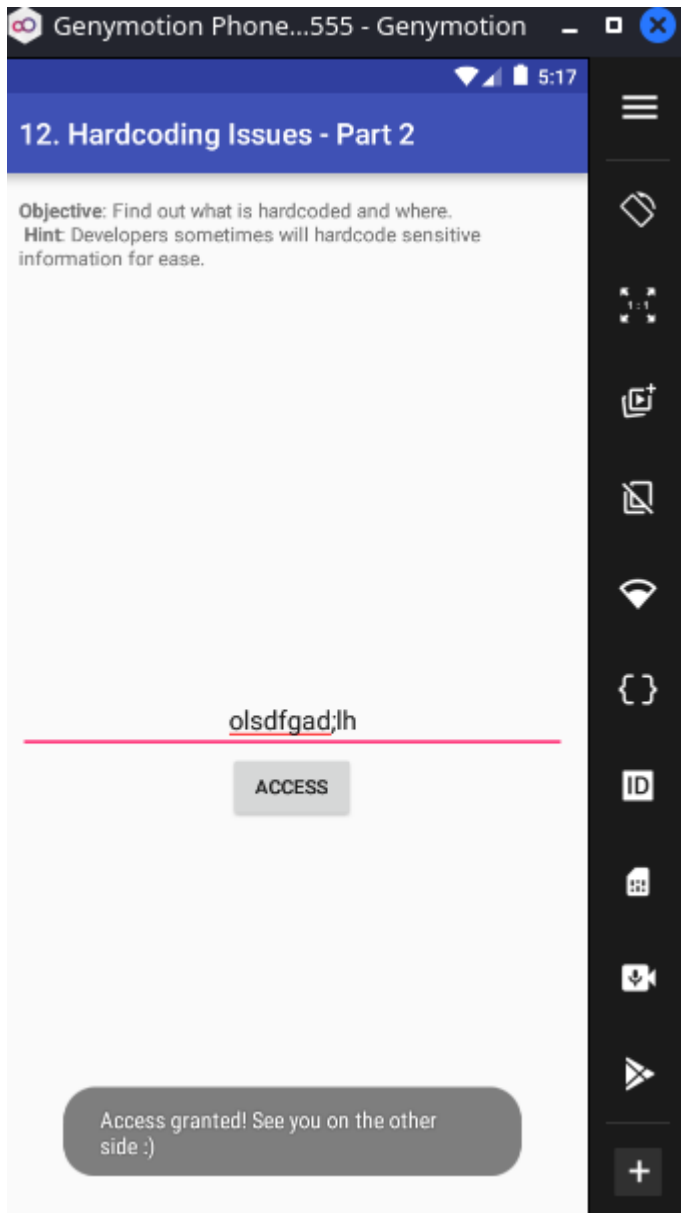
Explicación del fallo: El patrón vulnerable es: **Entrada del usuario** → **Java** → **JNI** → **librería nativa** → **comparación con secreto hardcodeado**. El problema principal es que la clave está dentro del cliente. Aunque no esté visible directamente en Java, sigue estando dentro del APK en la librería nativa **.so**. Mover un secreto desde Java a código nativo no lo hace seguro. Aunque **dex2jar** o **JD-GUI** no muestren la clave en una clase Java, el APK contiene la librería: **libdivajni.so**. Esa librería puede analizarse con herramientas como **strings**, **radare** o **cutter**.

Comprobamos en el dispositivo android:

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ find diva_apk -name "libdivajni.so"
diva_apk/lib/armeabi-v7a/libdivajni.so
diva_apk/lib/x86/libdivajni.so
diva_apk/lib/armeabi/libdivajni.so
diva_apk/lib/arm64-v8a/libdivajni.so
diva_apk/lib/x86_64/libdivajni.so
diva_apk/lib/mips/libdivajni.so
diva_apk/lib/mips64/libdivajni.so

(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$
```

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ strings diva_apk/lib/x86/libdivajni.so
__cxa_finalize
__cxa_atexit
__stack_chk_fail
Java_jakhar_aseem_diva_DivaJni_access
Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence
strcpy
JNI_OnLoad
_edata
__bss_start
_end
libstdc++.so
libm.so
libc.so
libdl.so
libdivajni.so
d$0[
olsdfgad;lh
.dotdot
;*2$"
```



La causa raíz es confiar en un secreto almacenado en el cliente. Mover la clave de Java a código nativo dificulta ligeramente el análisis, pero no impide su extracción. La mitigación consiste en no almacenar secretos dentro del APK y trasladar la validación a un backend o a un mecanismo donde el secreto no sea accesible desde el cliente.

Tabla resumen del Reto 12:

Campo	Resultado
Reto	Hardcoding Issues - Part 2
Clase principal	<code>Hardcode2Activity.java</code>
Clase JNI	<code>DivaJni.java</code>
Librería nativa	<code>libdivajni.so</code>
Fichero fuente nativo	<code>divajni.c</code>
Método Java relevante	<code>access(View view)</code>
Método JNI relevante	<code>DivaJni.access(String key)</code>
Campo leído	<code>R.id.hc2Key</code>
API relevante	<code>System.loadLibrary("divajni")</code>
Función nativa	<code>Java_jakhar_aseem_diva_DivaJni_access()</code>
Secreto expuesto	<code>olsdfgad;lh</code>
Tipo de fallo	Secreto hardcodeado en código nativo
Causa raíz	Clave almacenada dentro del APK en una librería <code>.so</code>
Fragmento vulnerable	<code>#define VENDORKEY "olsdfgad;lh"</code>
Cómo se comprueba	Extraer <code>libdivajni.so</code> y ejecutar <code>strings</code>
Comando útil	<code>find diva_unzip -name "libdivajni.so" -exec strings {} \;</code>
Resultado	La app concede acceso al introducir <code>olsdfgad;lh</code>
Impacto	Extracción de secretos mediante reversing de librería nativa
Mitigación	No guardar secretos en cliente. Validar en servidor. Usar mecanismos seguros de gestión de claves

2.13 Input Validation Issues — Part 3

Como ya se analizó y resolvió en el [Reto 13 de la Práctica 2](#), el objetivo ahora es analizar estáticamente el código responsable de la vulnerabilidad asociada a este reto: Una validación insegura de entrada que se realiza en código nativo. La aplicación pide un código para iniciar una “secuencia de lanzamiento”. Ese código se introduce en la app Android, pero se valida dentro de la librería nativa: `libdivajni.so`. El reto consiste en descubrir qué entrada permite superar la validación.

Clases y ficheros relevantes:

- `InputValidation3Activity`.
- `DivaJni`.
- `divajni.c`.
- `libdivajni.so`

Código en `InputValidation3Activity`: La clase Java principal es: `InputValidation3Activity`. En ella se crea un objeto `DivaJni` y se llama al método nativo `initiateLaunchSequence()`:

```
public void push(View view) {
    EditText cTxt = (EditText) findViewById(R.id.iv3CodeText);

    if (djni.initiateLaunchSequence(cTxt.getText().toString()) != 0) {
        Toast.makeText(this, "Launching in T - 10 ...",
            Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Access denied!", Toast.LENGTH_SHORT).show();
    }
}
```

donde:

- `djni.initiateLaunchSequence(cTxt.getText().toString())`: El texto introducido por el usuario se envía directamente a una función nativa. En el código oficial, `InputValidation3Activity` obtiene el valor del campo `R.id.iv3CodeText` y lo pasa a `djni.initiateLaunchSequence(...)`.

Código en DivaJni.java: La clase `DivaJni` carga la librería nativa:

```
private static final String soName = "divajni";

static {
    System.loadLibrary(soName);
}

public native int initiateLaunchSequence(String code);
```

donde:

- `System.loadLibrary(soName);`: Hace que Android cargue la librería: `libdivajni.so`

El código de `DivaJni` declara el método nativo `initiateLaunchSequence(String code)` y carga la librería `divajni`.

Código vulnerable en divajni.c: La lógica importante está en el fichero nativo: `divajni.c`. Ahí se define el código correcto:

```
#define CODE ".dotdot"
#define CODESIZEMAX 20
```

Y después aparece la función:

```
JNIEXPORT jint JNICALL
Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence(
    JNIEnv * env,
    jobject jobj,
    jstring jcode
) {
    const char * pcode = (*env)->GetStringUTFChars(env, jcode, 0);
    int ret = 0;
    char code[CODESIZE_MAX];

    strcpy(code, pcode);

    if (code[0] == '!') {
        code[0] = '.';
    }

    ret = strncmp(CODE, code, sizeof(CODE) - 1);

    if (ret == 0) {
        ret = 1;
    } else {
        ret = 0;
    }

    return ret;
}
```

El código define `CODE` como `.dotdot`, reserva un buffer local `char code[CODESIZE_MAX]`, copia la entrada con `strcpy(code, pcode)`, cambia el primer carácter de `!` a `.`, y compara el resultado con `CODE`.

Explicación del fallo: El patrón vulnerable es: `Entrada del usuario` → `Java` → `JNI` → `strcpy()` → `modificación del primer carácter` → `comparación con código secreto`.

La aplicación espera comparar la entrada con: `.dotdot`. Pero antes de comparar, hace esto:

```
if (code[0] == '!') {
    code[0] = '.';
}
```

Eso significa que si el usuario introduce: `!dotdot`, el código nativo lo transforma internamente en: `.dotdot`.

Después se compara contra: `#define CODE ".dotdot"`, y la validación se supera.

Por qué es vulnerable: Hay dos problemas importantes:

- Problema 1 — Validación manipulable: La validación modifica la entrada del usuario antes de compararla:

```
if (code[0] == '!') {  
    code[0] = '.';  
}
```

donde:

- Esto permite que una entrada aparentemente incorrecta, !dotdot, se convierta en el código válido .dotdot.
- Problema 2 — Uso inseguro de strcpy():

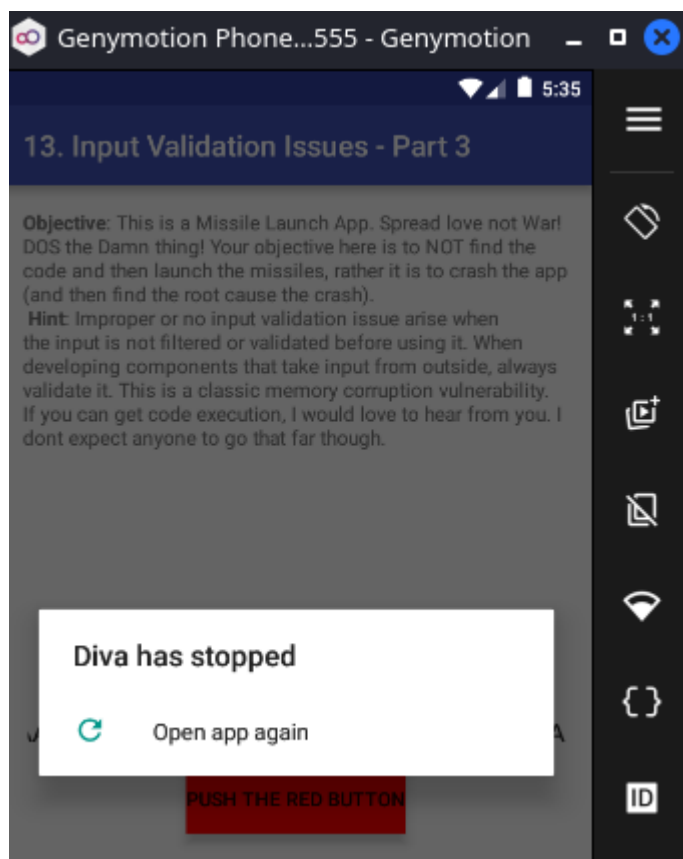
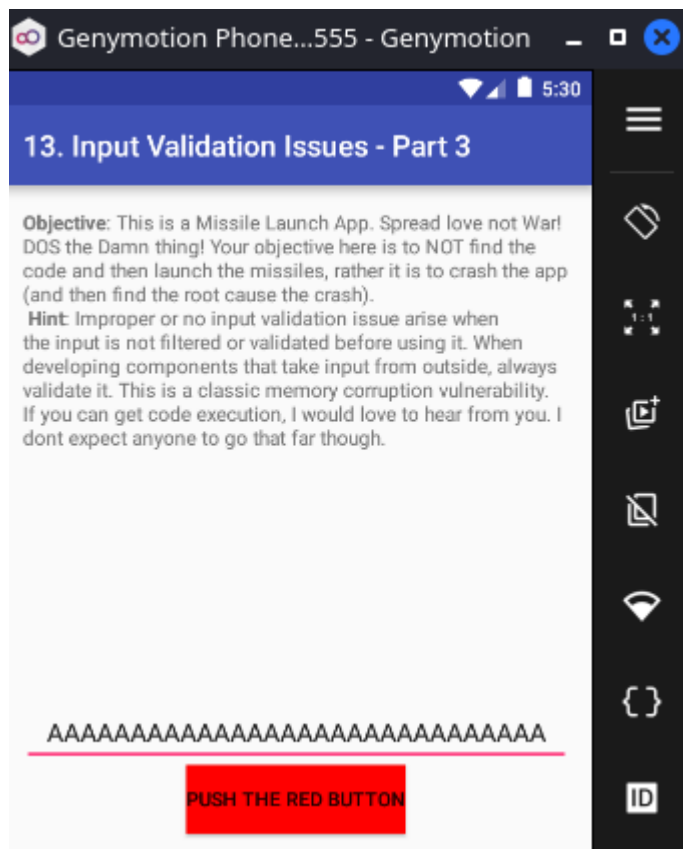
```
strcpy(code, pcode);  
...  
char code[CODESIZEMAX];  
...  
#define CODESIZEMAX 20  
...
```

donde:

- Copia la entrada del usuario en un buffer fijo:
- El problema es que strcpy() no comprueba si la entrada cabe dentro del buffer. Si el usuario introduce una cadena demasiado larga, podría provocar corrupción de memoria o un crash.
- Por tanto, el fallo no es solo lógico, sino también de seguridad en memoria.

Comprobamos en el dispositivo android:

```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]  
$ adb logcat -c  
  
(xxniwexx@kali)-[~/Escritorio/cybersecurity]  
$ adb logcat | grep -iE "fatal|exception|diva|crash"
```



```
(xxniwexx@kali)-[~/Escritorio/cybersecurity]
$ adb logcat | grep -iE "fatal|exception|diva|crash"
----- beginning of crash -----
05-06 05:35:01.922 5417 5417 F libc : Fatal signal 11 (SIGSEGV), code 1, fault addr 0x41414141 in tid 5417 (khar.aseem.diva)
05-06 05:35:01.983 5914 5914 F DEBUG : pid: 5417, tid: 5417, name: khar.aseem.diva >>> jakhar.aseem.diva <<<
05-06 05:35:02.071 5417 5417 F libc : Fatal signal 11 (SIGSEGV), code 2, fault addr 0xf230441c in tid 5417 (khar.aseem.diva)
05-06 05:35:02.072 5417 5417 F libc : Fatal signal 11 (SIGSEGV), code -6, fault addr 0x85 in tid 5417 (khar.aseem.diva)
05-06 05:35:02.072 578 5919 W ActivityManager: Force finishing activity jakhar.aseem.diva/.InputValidation3Activity
05-06 05:35:02.577 578 592 W ActivityManager: Activity pause timeout for ActivityRecord{6426305 u0 jakhar.aseem.diva/.InputValidation3Activity t32 f}
```

La causa raíz es una validación de entrada insegura implementada en código nativo. La mitigación consiste en validar la longitud de la entrada antes de copiarla, evitar funciones inseguras como `strcpy()`, eliminar transformaciones ambiguas de la entrada y no confiar en lógica sensible implementada únicamente en el cliente.

Tabla resumen del Reto 13:

Campo	Resultado
Reto	Input Validation Issues - Part 3
Clase principal	<code>InputValidation3Activity.java</code>
Clase JNI	<code>Divajni.java</code>
Librería nativa	<code>libdivajni.so</code>
Fichero fuente nativo	<code>divajni.c</code>
Método Java relevante	<code>push(View view)</code>
Método JNI relevante	<code>Divajni.initiateLaunchSequence(String code)</code>
Campo leído	<code>R.id.ivi3CodeText</code>
API relevante	<code>System.loadLibrary("divajni")</code>
Constante comparada	<code>#define CODE ".dotdot"</code>
Buffer usado	<code>char code[CODESIZEMAX]</code>
Tamaño del buffer	20 bytes
Función problemática	<code>strcpy(code, pcode)</code>
Tipo de fallo	Validación de entrada insegura en código nativo
Causa raíz	Se modifica la entrada del usuario antes de compararla y se usa <code>strcpy()</code> sin control de longitud
Fragmento vulnerable	<code>if (code[0] == '!') { code[0] = '.'; }</code>
Código válido para el reto	<code>!dotdot</code>
Resultado	Se muestra <code>Launching in T - 10 ...</code>
Impacto	Bypass de validación y posible corrupción de memoria con entradas largas
Mitigación	Validar longitud, usar funciones seguras, eliminar transformaciones ambiguas y no confiar en validaciones críticas del lado cliente